

Wykład 10

Uczenie ze wzmocnieniem

Metody wyznaczania optymalnych strategii

RL - *Reinforcement learning*

Podstawowe pojęcia

- Nagroda (ang. Reward):

Wartość zwracana przez środowisko w momencie wykonania akcji wybranej przez agenta.

Reprezentuje cel, lub cele, jakie agent ma osiągnąć.

Oznaczenie: r_t - nagroda natychmiastowa otrzymana w chwili czasu t .

Funkcja nagrody:

funkcja, która zapewnia informacje zwrotne na temat wartości wynikających a podjętych akcji, kierując agenta w stronę jego celu.

Podstawowe pojęcia

- Oczekiwana nagroda (ang. Return):

Oczekiwana nagroda po zakończeniu bieżącego epizodu.

Celem uczenia za wzmocnieniem jest maksymalizacja oczekiwanej nagrody.

Oznaczenie:

Oczekiwana przyszła nagroda w chwili t :

$$G_t = R_{t+1} + R_{t+2} + \dots = \sum_{i=t+1}^{\infty} R_i$$

Zdyskontowana oczekiwana nagroda:

$$R_t = \sum_{i=t+1}^{\infty} \gamma^i R_i, \text{ gdzie } \gamma \in [0,1), \text{ discount factor, czynnik dyskontujący}$$

γ jest bardzo ważny. Odgrywa rolę kary za długość trajektorii dojścia do celu.

Przykłady ćw. 11

Podstawowe pojęcia

- **Funkcja wartości (oceny, ang. Value function):**

Określa, jak dobrze być w danym stanie.

Szacuje przyszłe skumulowane nagrody, które agent otrzyma z danego stanu.

Oczekiwana nagroda (zdyskontowana suma nagród), jaką możemy otrzymać, rozpoczynając w stanie s i działając zgodnie ze strategią π .

- Rodzaje:

- Funkcja wartości stanu (wartość stanu, state value)**

- $V^\pi(s)$ - jak dobrze być w stanie s , działając zgodnie ze strategią π .

- Funkcja wartości stanu-akcji (wartość akcji, action value)**

- $Q^\pi(s, a)$ - jak dobrze będąc w stanie s jest wykonać akcję a , a potem działać zgodnie ze strategią π , q-value

Funkcja wartości stanu-akcji (Funkcja Q)

$$Q(s_t, a_t) = E[R_t | s_t, a_t]$$

Oczekiwana nagroda, jaką możemy otrzymać, rozpoczynając w stanie s i podejmując akcję a : $q(s, a)$

Strategia polega na wyborze akcji, która maksymalizuje oczekiwaną nagrodę (przyszłą nagrodę)

$$\pi^*(s) = \underset{a}{\operatorname{arg\,max}} Q(s, a)$$

Zależność między $v(s)$ a $q(s, a)$

Wartość stanu jest średnią ważoną wartości akcji, zgodnie z prawdopodobieństwami wyboru akcji w strategii π :

- $v_{\pi}(s) = \sum_a \pi(a|s) q_{\pi}(s, a)$

$v_{\pi}(s)$ mówi, jak dobry jest stan przy polityce π

$q_{\pi}(s, a)$ mówi, jak dobra jest dobra konkretna akcja w tym stanie.

UWAGA!

Wybieramy akcję w stanie s , a następnie kierujemy się strategią π

Definicja

Niech S będzie zbiorem skończonym elementów, które nazywamy stanami. Niech wektor $p = [p_i]_{i \in S} = [p(i)]_{i \in S}$ będzie rozkładem prawdopodobieństwa na S . Atomy tego rozkładu nazywają się prawdopodobieństwami początkowymi, zaś wektor p rozkładem początkowym.

Przez $P = [p(ij)]_{i,j \in S} = [p_{ij}]_{i,j \in S}$ oznaczmy macierz stochastyczną, której elementy p_{ij} oznaczają prawdopodobieństwo przejścia ze stanu i do stanu j , $p_{ij} = P(X_{n+1} = j | X_n = i)$.

Macierz $P = [p_{ij}]$ nazywamy macierzą przejścia łańcucha Markowa.

Macierz $P = [p_{ij}]$ jest macierzą stochastyczną, tzn. $\sum_j p_{ij} = 1$.

$$P = \begin{pmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ p_{31} & p_{32} & p_{33} \end{pmatrix}$$

Własność Markowa

Mówimy, że ciąg zmiennych losowych X_n , $n = 0, 1, 2, \dots$, o wartościach w S jest (jednorodnym, skończonym) łańcuchem Markowa, jeżeli spełnione są następujące warunki:

1. $P(X_0 = i) = p(i)$, dla każdego $i \in S$,

2. dla każdego $n \geq 0$ zachodzi równość:

$$P(X_{n+1} = j | X_0 = i_0, \dots, X_n = i) = P(X_{n+1} = j | X_n = i) = p(i, j) = p_{ij}, \text{ dla wszystkich } i_0, \dots, i, j \in S.$$

Andriej Markow

Андрей Андреевич Марков



1856-1922

Własność Markowa

Własność Markowa:

Stan ŁM w chwili $n+1$ zależy wyłącznie od stanu w chwili n .

$$P(X_{n+1} = j | X_0 = i_0, \dots, X_n = i) = P(X_{n+1} = j | X_n = i)$$

Założenie jednorodności:

Prawdopodobieństwa przejścia są niezależne od n : $P(X_{n+1} = j | X_n = i) = p(i, j) = p_{ij}$.

Przyszłość jest niezależna od przeszłości, biorąc pod uwagę teraźniejszość”

Stan przechwytuje wszystkie istotne informacje z historii

Gdy stan jest znany, historia może zostać odrzucona

Stan jest wystarczającą statystyką przyszłości

Przykład

Łańcuch pogodowy

Założmy, że latem w Warszawie można zaobserwować następującą pogodę S (słonecznie) Z (zachmurzenie), D (deszcz). Założmy, że meteorolog obserwował pogodę przez kilka dni i zanotował następujący ciąg: {S,S,Z,D,D,Z,S,S,S,Z,S,Z,Z,D,Z,S,Z,S,S,D,Z,Z,D,D}.

Chcemy odpowiedzieć na następujące pytanie:

Zakładając, że dziś jest słonecznie, oblicz jakie jest prawdopodobieństwo słonecznej pogody jutro.

Mamy następujące przejścia:

S-S: 4:

S-Z: 4

S-D: 1

Z-S: 4

Z-Z: 2

Z-D: 3

D-S: 0

D-Z: 3

D-D: 2

Mamy następującą macierz częstości przejść:

	S	Z	D	Razem
S	4	4	1	9
Z	4	2	3	9
D	0	3	2	5

Macierz przejścia jest postaci:

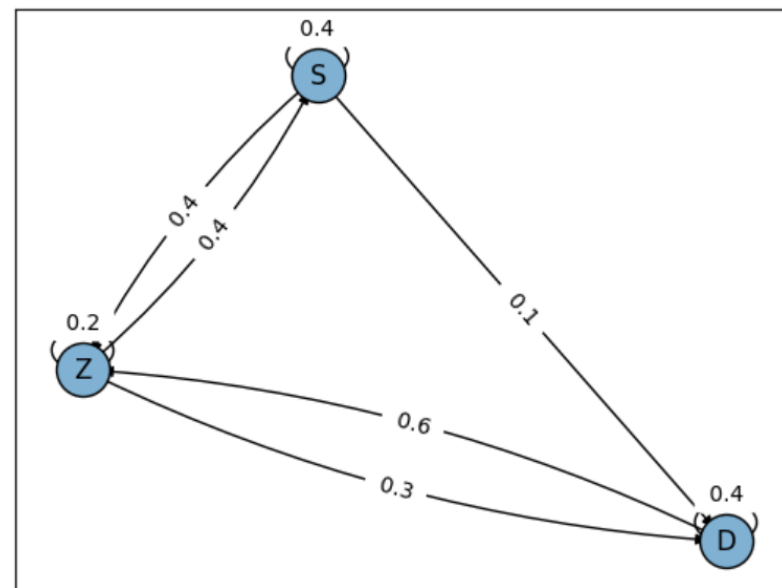
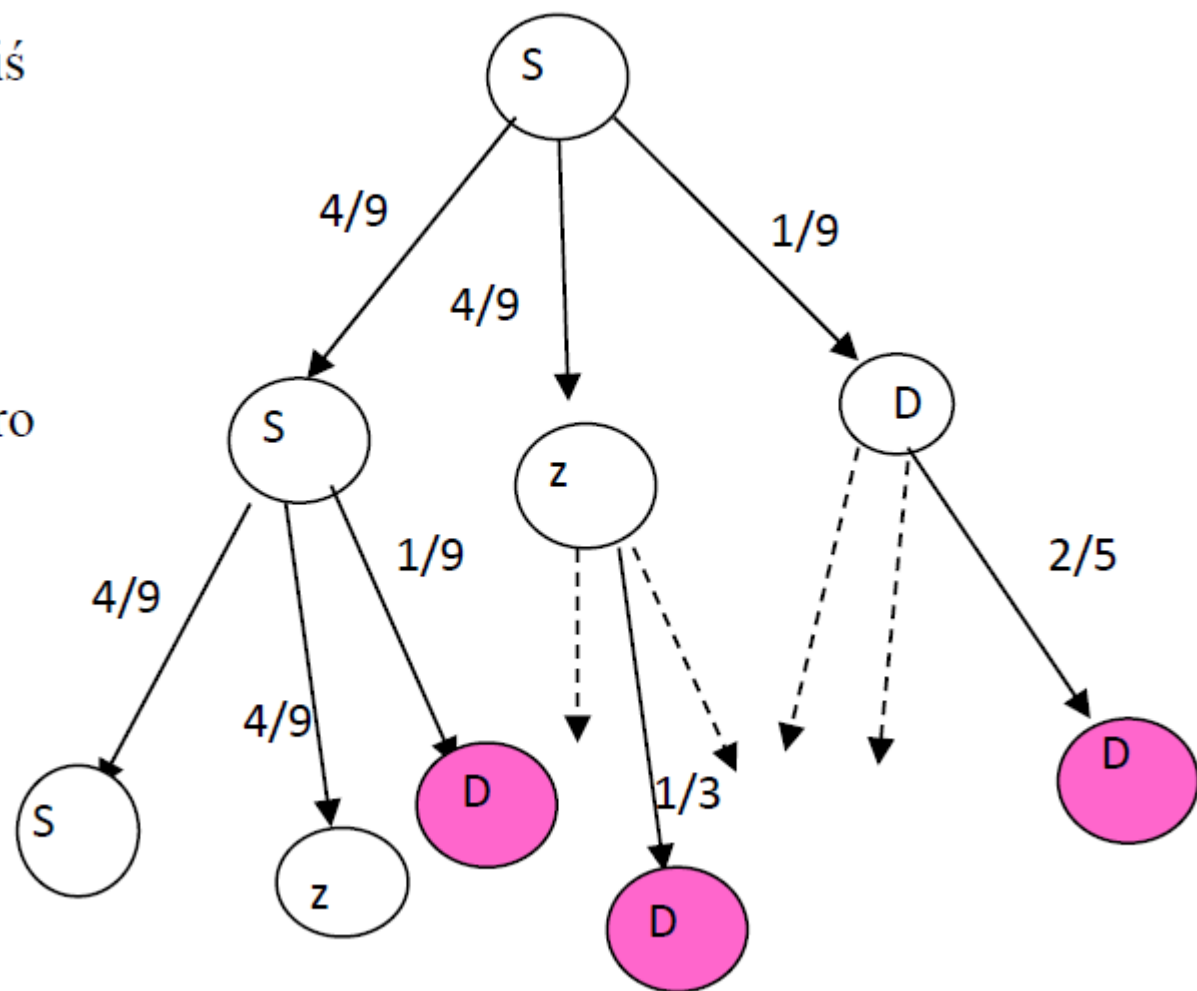
$$P = \begin{matrix} & \begin{matrix} S \\ Z \\ D \end{matrix} \\ \begin{matrix} S \\ Z \\ D \end{matrix} & \begin{pmatrix} 4/9 & 4/9 & 1/9 \\ 4/9 & 2/9 & 1/3 \\ 0 & 3/5 & 2/5 \end{pmatrix} \end{matrix}$$

Diagram

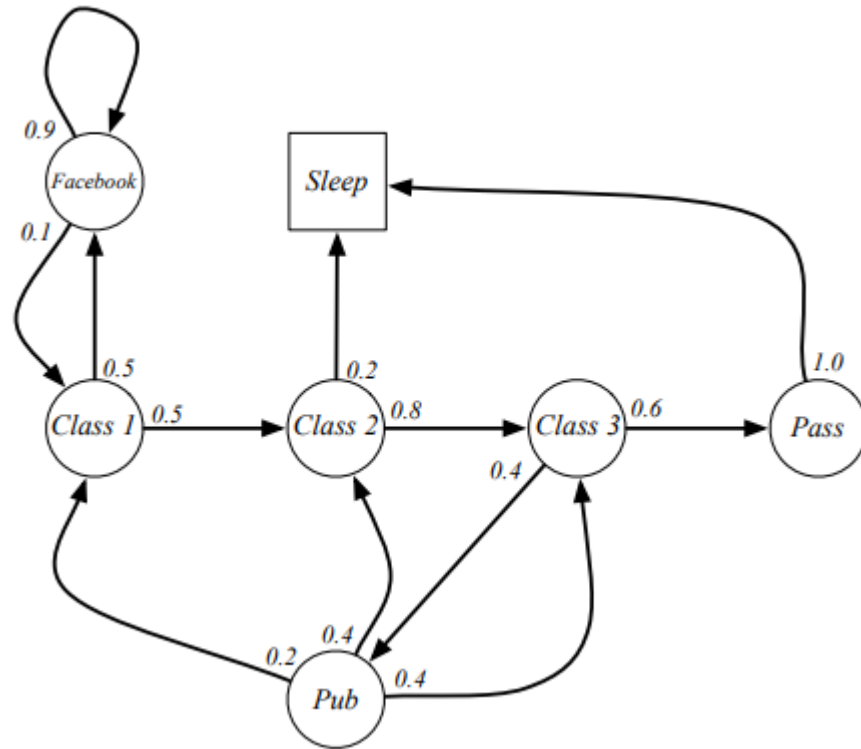
Dziś

Jutro

Pojutrze



Macierz przejścia ŁM z wykładu D. Silver



$$P = \begin{bmatrix} & C1 & C2 & C3 & Pass & Pub & FB & Sleep \\ s & & 0.5 & & & & 0.5 & \\ & & & 0.8 & & & & 0.2 \\ b & & & & 0.6 & 0.4 & & 1.0 \\ p & 0.2 & 0.4 & 0.4 & & & 0.9 & \\ & 0.1 & & & & & & 1 \end{bmatrix}$$

Proces Markowa z nagrodami/wartościami (MRP)

Definicja

Proces Markowa z nagrodami to krotka $\langle S, P, R, \gamma \rangle$, gdzie S -skończony zbiór stanów;

P -macierz migracji (transition matrix),

$$p_{ss'} = P(X_{t+1} = s' | X_t = s).$$

Funkcja nagrody R_s

$R_s = E(R_{t+1} | X_t = s)$, $\gamma \in [0,1]$, czynnik dyskontujący

Łańcuch (proces) Markowa z wartościami

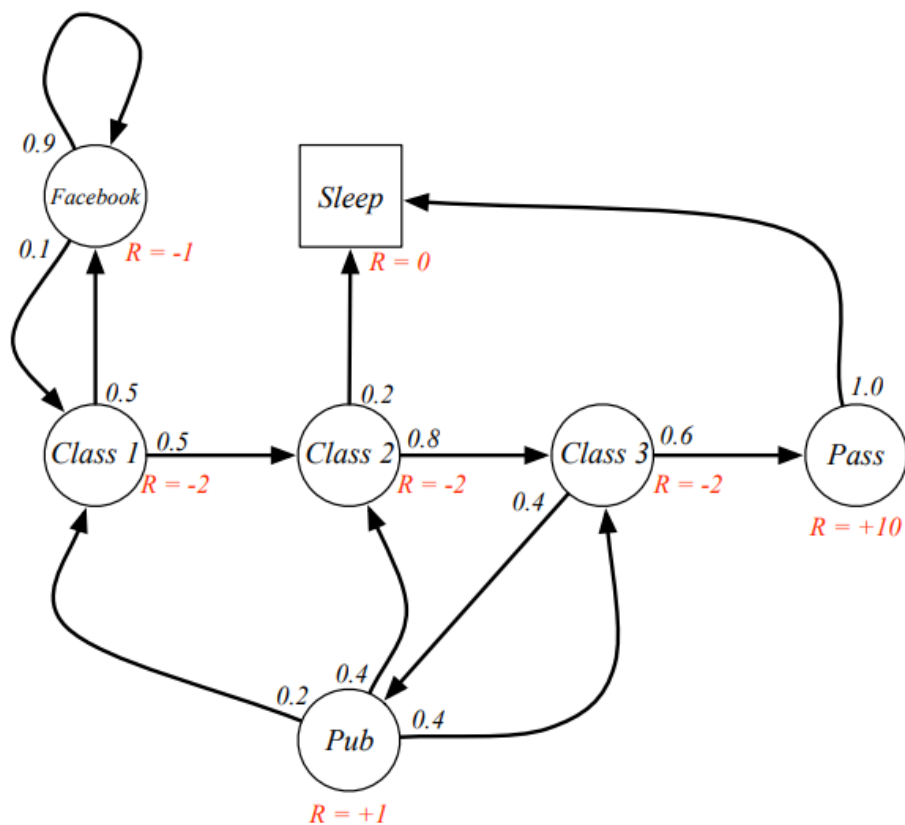
Niech $r(s)$ lub $R(s)$ oznacza nagrodę natychmiastową otrzymywaną w stanie s .

G_t – oczekiwana (przyszła) nagroda w chwili t

$$G_t = R_{t+1} + R_{t+2} + \dots = \sum_{i=t+1}^{\infty} R_i$$

Zdyskontowana oczekiwana nagroda:

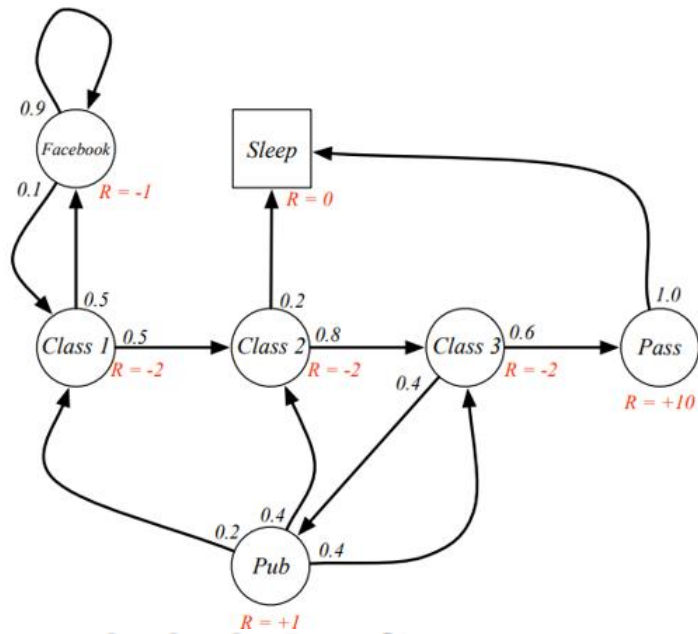
$$R_t = \sum_{i=t+1}^{\infty} \gamma^i R_i, \text{ gdzie } \gamma \in [0,1], \text{ discount factor}$$



$$G_t = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

G_t i R_t są zmiennymi losowymi

Użyteczności stanu C1 z gamma=0.5



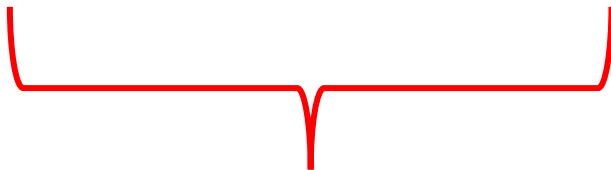
C1 C2 C3 Pass Sleep

C1 FB FB C1 C2 Sleep

C1 C2 C3 Pub C2 C3 Pass Sleep

C1 FB FB C1 C2 C3 Pub C1 ...

FB FB FB C1 C2 C3 Pub C2 Sleep



$$G_t = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

$$v_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 10 * \frac{1}{8} = -2.25$$

$$v_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16} = -3.125$$

$$v_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 1 * \frac{1}{8} - 2 * \frac{1}{16} \dots = -3.41$$

$$v_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16} \dots = -3.20$$

$$\begin{aligned} G_t &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \\ &= R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

Epizody, próby o skończonej liczbie kroków, trajektorie ŁM

Procesy Decyzyjne Markowa

MDP to ŁM z nagrodami, w którym wprowadzamy decyzje/akcje.

Jest to środowisko, w którym wszystkie stany mają własność Markowa.

$$M = \{S, A, P, R, \gamma\}$$

Zbiory:

S – przestrzeń stanów;

A – przestrzeń działań, akcji, $A(s)$ -zbiór działań dla stanu $s \in S$

R – zbiór nagród; $R(s, a)$ -nagrody dla każdej pary (s, a)

(lub $r(s, a, s')$ nagroda natychmiastowa za migrację z s do s' za pomocą działania a)

Proces Decyzyjny Markowa

Model/dynamika

P—operator migracji/przejścia (state transition operator, tensor)

$$P(s, a, s') = p(s'|s, a)$$

$$\sum_{s' \in S} p(s'|s, a) = 1 \text{ dla każdej pary } (s, a)$$

Prawdopodobieństwo nagrody r , dla stanu s i akcji a : $p(r|s, a)$

$$\sum_{r \in R(s, a)} p(r|s, a) = 1$$

Wartość stanu (state value: V, v)

Funkcja wartości stanu $V_\pi(s)$ - jak dobrze być w stanie s , działając zgodnie ze strategią π .

Szacuje przyszłe skumulowane nagrody, które agent otrzyma z danego stanu.

Oczekiwana suma nagród, jaką możemy otrzymać, rozpoczynając w stanie s i działając zgodnie ze strategią π .

$$V_\pi(s) = E(G_t | X_t = s)$$

G_t i R_t są zmiennymi losowymi

Strategia/Policy

Strategia, to odwzorowanie $\pi: S \rightarrow A$, z przestrzeni stanów na przestrzeń akcji.

Prawdopodobieństwo wyboru akcji a w stanie s jest równe $\pi(a|s)$

$$\pi(a|s) = P(A_t = a | X_t = s)$$

$$\sum_{a \in A(s)} \pi(a|s) = 1$$

$$\mathcal{P}_{s,s'}^\pi = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{P}_{ss'}^a$$

$$\mathcal{R}_s^\pi = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{R}_s^a$$

Strategia mówi agentowi jaką akcję ma wybrać w każdym stanie.

Strategia deterministyczna: $a = \pi(s)$

Strategia stochastyczna: $\pi(a|s) = P(a_t = a | X_t = s)$

Własność Markowa

Własność Markowa dotyczy braku pamięci dla procesu dotyczącego kolejnych nagród oraz procesu dotyczącego zmiany stanu.

Mamy następujące trajektorie (epizody):

$s_0, a_0, r_1, s_1, a_1, r_2, s_2, \dots, r_T, s_T$, s_T -stan terminalny

$$p(s_{t+1}|s_t, a_t, s_{t-1}, \dots, s_0, a_0) = p(s_{t+1}|s_t, a_t)$$

$$p(r_{t+1}|s_t, a_t, s_{t-1}, \dots, s_0, a_0) = p(r_{t+1}|s_t, a_t)$$

Ze względu na własność Markowa, MDP może być całkowicie opisany przez:

- **Funkcję nagrody** $r : S \times A \rightarrow \mathbb{R}$

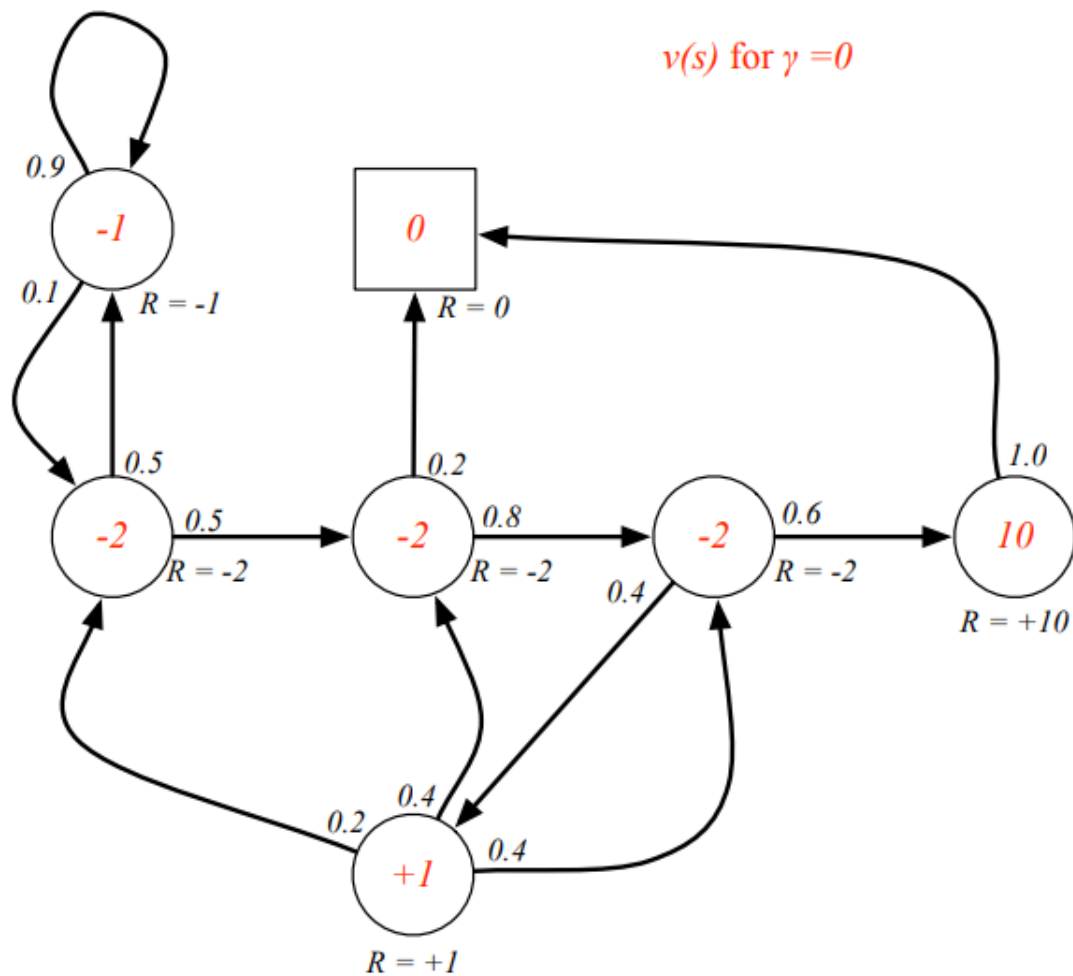
$r_a(s)$ = **natychmiastowa** nagroda, jeśli agent jest w stanie s i podejmuje działanie a . Jest to nagroda **krótkoterminowa**.

- **Model przejścia** (dynamika procesu):

$$P: S \times A \times S \rightarrow [0, 1]$$

prawdopodobieństwo przejścia z s do s' pod warunkiem podjęcia akcji/działania a

$$P(s, a, s') = p(s'|s, a)$$

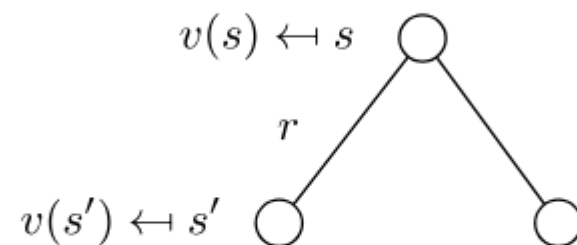
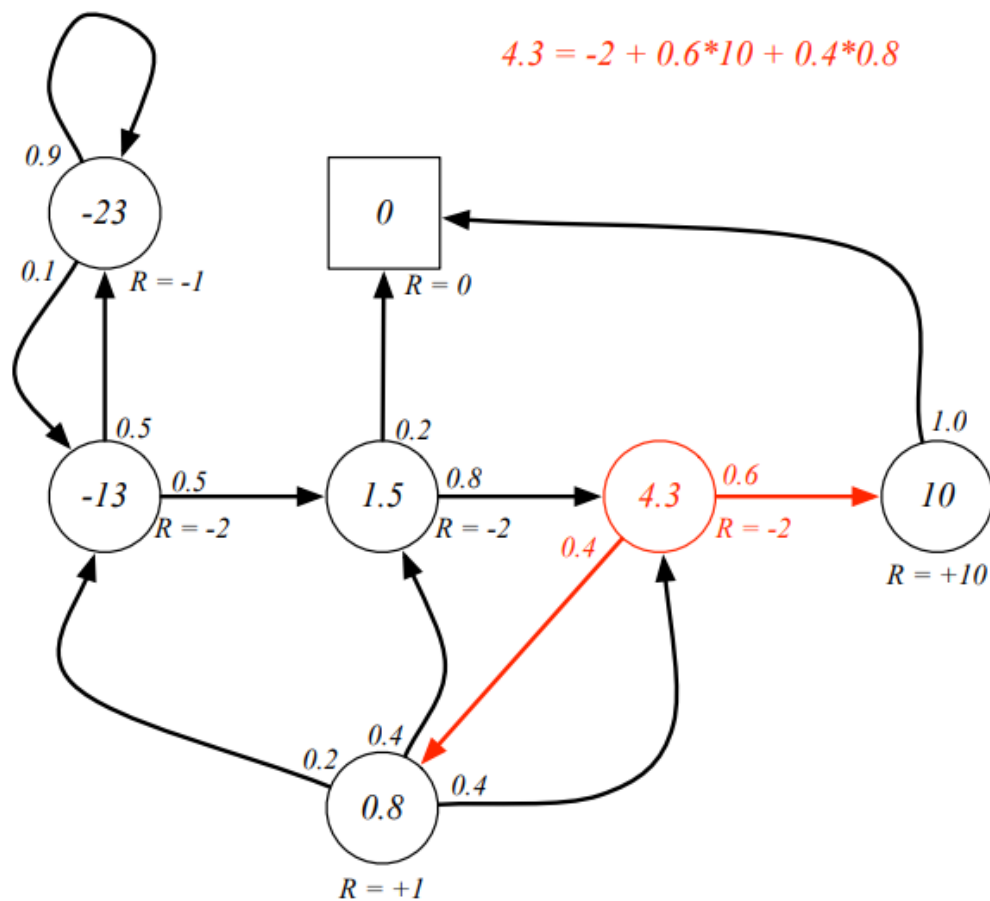


Funkcja wartości stanu $v(s)$ procesu Markowa z nagrodami jest oczekiwaną nagrodą /użytecznością z bycia w stanie s .

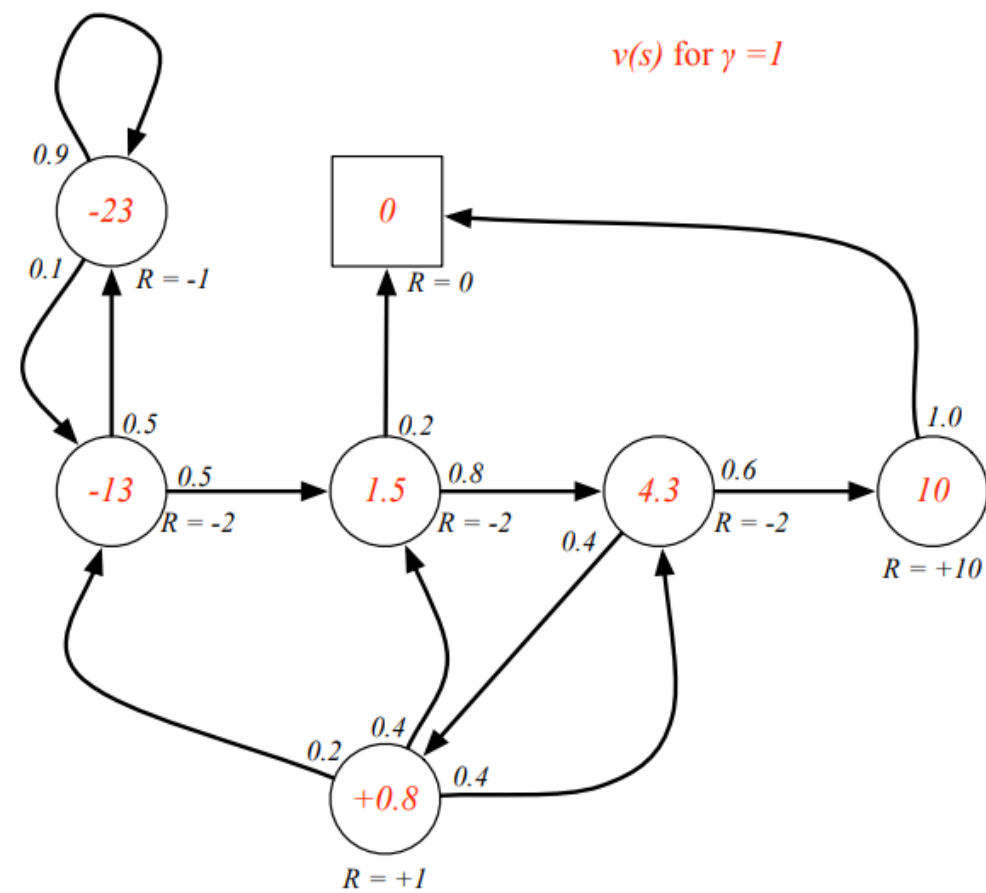
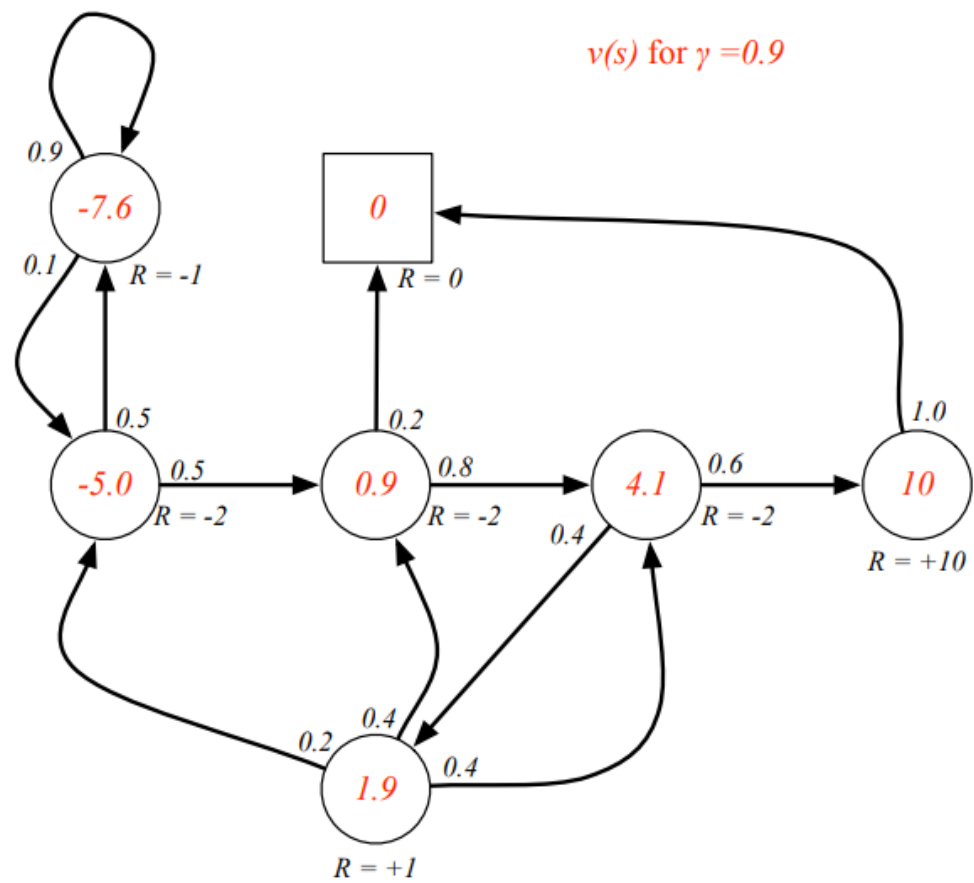
$$v(s) = \mathbb{E}[G_t \mid S_t = s]$$

Równanie Bellmana

$$v(s) = \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s]$$



$$v(s) = \mathcal{R}_s + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'} v(s')$$



Funkcja wartości stanu $V^\pi(s)$ (wartość stanu)

$V^\pi(s)$ zależy od s oraz π , ale nie zależy od czasu t .

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

$$= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots)$$

$$= R_{t+1} + \gamma G_{t+1},$$

$$G_{t+1} = R_{t+2} + \gamma R_{t+3} + \dots$$

$$V^\pi(s) = v_\pi(s) = \mathbb{E}[G_t | S_t = s]$$

$$= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s]$$

$$= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[G_{t+1} | S_t = s].$$

Funkcja wartości stanu

$$\begin{aligned}v(s) &= \mathbb{E}[G_t \mid S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s]\end{aligned}$$

$$v(s) = \mathbb{E}[G_t \mid S_t = s]$$

Funkcja wartości stanu rozkłada się na 2 części

- Nagrodę natychmiastową R_{t+1}
- Zdyskontowaną wartość stanu następnego

Funkcja wartości stanu $V^\pi(s)$ (wartość stanu)

$$\begin{aligned}v_\pi(s) &= \mathbb{E}[G_t | S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s] \\&= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[G_{t+1} | S_t = s].\end{aligned}$$

Wartość oczekiwana nagrody natychmiastowej

$$\begin{aligned}\mathbb{E}[R_{t+1} | S_t = s] &= \sum_{a \in \mathcal{A}} \pi(a|s) \mathbb{E}[R_{t+1} | S_t = s, A_t = a] \\&= \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a) r.\end{aligned}$$

Funkcja wartości stanu $V^\pi(s)$ (wartość stanu)

Wartość oczekiwana nagrody przyszłej

$$\begin{aligned}\mathbb{E}[G_{t+1}|S_t = s] &= \sum_{s' \in \mathcal{S}} \mathbb{E}[G_{t+1}|S_t = s, S_{t+1} = s']p(s'|s) \\ &= \sum_{s' \in \mathcal{S}} \mathbb{E}[G_{t+1}|S_{t+1} = s']p(s'|s) \quad \text{Własność Markowa} \\ &= \sum_{s' \in \mathcal{S}} v_\pi(s')p(s'|s) \\ &= \sum_{s' \in \mathcal{S}} v_\pi(s') \sum_{a \in \mathcal{A}} p(s'|s, a)\pi(a|s).\end{aligned}$$

Funkcja wartości stanu $V^\pi(s)$ (wartość stanu)

$$\begin{aligned} v_\pi(s) &= \mathbb{E}[R_{t+1}|S_t = s] + \gamma \mathbb{E}[G_{t+1}|S_t = s], \\ &= \underbrace{\sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a) r}_{\text{oczekiwana nagród natychmiastowych}} + \underbrace{\gamma \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s, a) v_\pi(s')}_{\text{oczekiwana nagród przyszłych}} \\ &= \sum_{a \in \mathcal{A}} \pi(a|s) \left[\sum_{r \in \mathcal{R}} p(r|s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v_\pi(s') \right] \quad \text{dla wszystkich } s \in \mathcal{S} \end{aligned}$$

Równania Bellmana

$$v_{\pi}(s) = r_{\pi}(s) + \gamma \sum_{s' \in \mathcal{S}} p_{\pi}(s'|s) v_{\pi}(s'),$$

$$r_{\pi}(s) \doteq \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a) r,$$

$$p_{\pi}(s'|s) \doteq \sum_{a \in \mathcal{A}} \pi(a|s) p(s'|s, a).$$

$$v_{\pi} = r_{\pi} + \gamma P_{\pi} v_{\pi},$$

Równania Bellmana dla MDP (macierzowe)

- Wyznaczanie wartości stanów z równań Bellmana dla strategii π

$$v_{\pi} = \mathcal{R}^{\pi} + \gamma \mathcal{P}^{\pi} v_{\pi}$$

$$v_{\pi} = (I - \gamma \mathcal{P}^{\pi})^{-1} \mathcal{R}^{\pi}$$

Równania Bellmana oceniają skuteczność strategii poprzez oszacowanie oczekiwanej wartości wynikającej z jej zastosowania.

Równania Bellmana pozwalają wyznaczyć wartości stanów dla konkretnej strategii.

Równania Bellmana dla procesu z nagrodami

$$v = \mathcal{R} + \gamma \mathcal{P}v$$

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}_1 \\ \vdots \\ \mathcal{R}_n \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{n1} & \dots & \mathcal{P}_{nn} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

Znany jest głównie jako twórca programowania dynamicznego (Dynamic Programming 1957).

Od lat 40. XX wieku, pomiędzy rokiem 1948 i 1952, Richard Bellman rozwijał teorię programowania dynamicznego. Pracując na wydziale matematyki i w RAND, badał problem określania alokacji pocisków do celów tak aby spowodować możliwie największe szkody. Praca ta doprowadziła go do sformułowania zasady optymalności programowania dynamicznego. W 1957 roku Richard Bellman zastosował programowania dynamicznego do sterowania optymalnego układami dyskretnymi, co ukazało jednocześnie, naturalny zwrot w podejściu do rozwiązywania problemów sterowania.



Rozwiązanie równań Bellmana

$$v = \mathcal{R} + \gamma \mathcal{P}v$$

$$(I - \gamma \mathcal{P})v = \mathcal{R}$$

$$v = (I - \gamma \mathcal{P})^{-1} \mathcal{R}$$

Złożoność obliczeniowa $O(n^3)$, n – liczba stanów

Metody rozwiązywania:

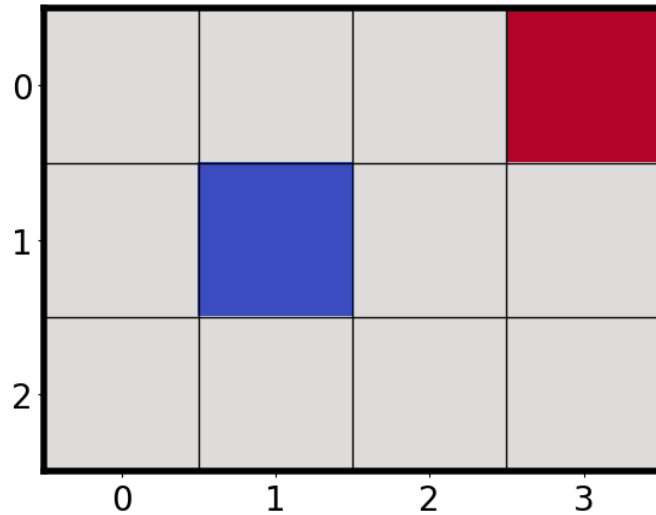
- Dynamiczne;
- Monte Carlo;
- Różnic czasowych (TD)

Richard Ernest Bellman



1920 –1984

Przykład



```
# Move right:
elif state_x < n_x-1 and state_x > 0:
    next_state = (state_y, state_x + 1)
elif state_x == 0 and state_y == 0:
    next_state = (state_y, state_x + 1)

# Move up:
elif state_x == n_x-1:
    next_state = (state_y - 1, state_x)
elif state_x == 0 and state_y > 0:
    next_state = (state_y - 1, state_x)

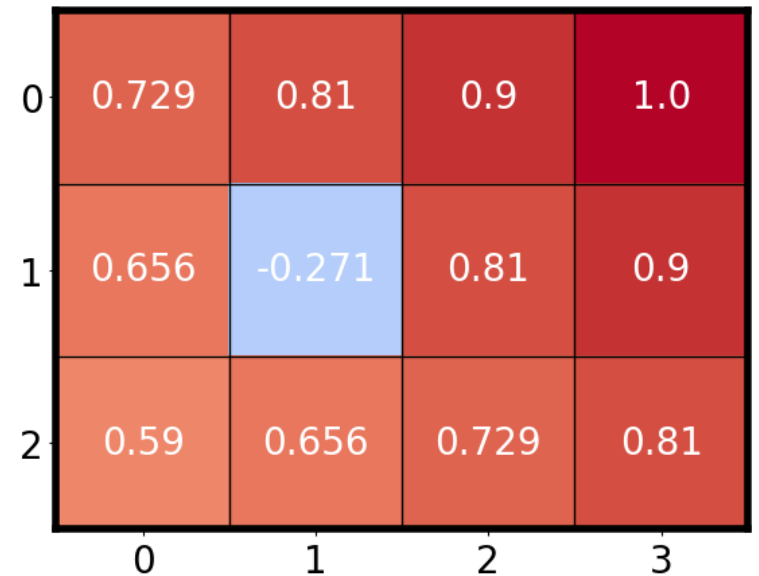
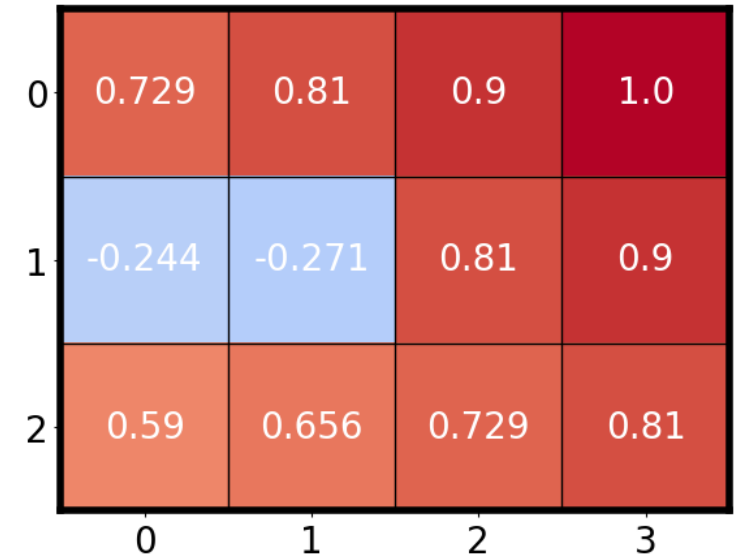
return next_state
```

- Strategia 1

Dla wszystkich stanów z wyjątkiem ostatniej kolumny idziemy w prawo. W ostatniej kolumnie w górę.

- Strategia 2

Omijamy stan zabroniony



Cel: Znalezienie strategii (maksymalizującej nagrodę)

Strategia to może być sieć neuronowa, albo tablica dla dyskretnej przestrzeni stanów i akcji.

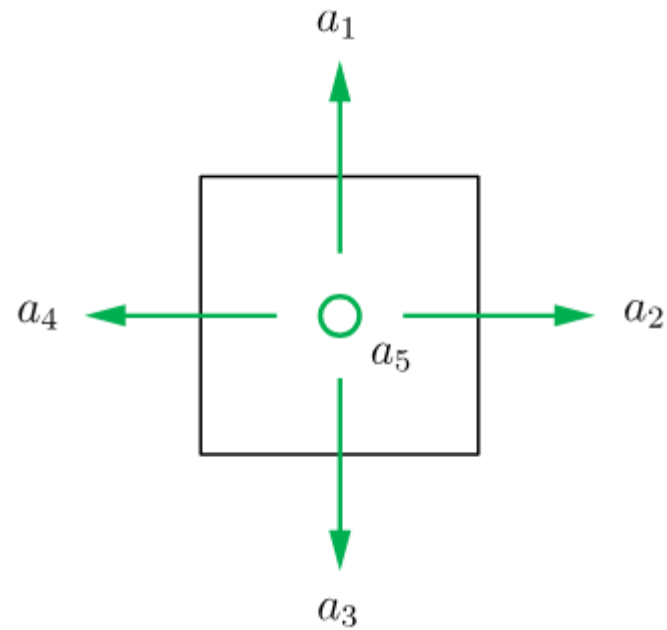
Nie zawsze dany jest model.

Dla danego modelu mamy 2 algorytmy:

- Iteracja wartości
- Iteracja strategii

Przykład

s_1	s_2	s_3
s_4	s_5	s_6
s_7	s_8	s_9

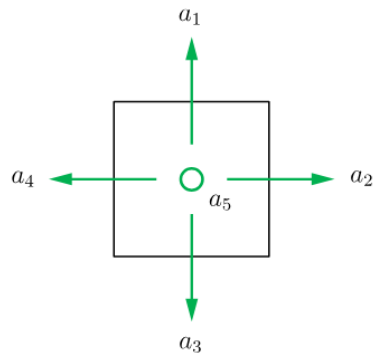


5 możliwych akcji.

Przykład środowiska na siatce. Mamy 9 stanów. Pomarańczowe stany są zabronione (forbidden).

Przykład

s_1	s_2	s_3
s_4	s_5	s_6
s_7	s_8	s_9



$$p(s_1|s_1, a_2) = 0$$

$$p(s_2|s_1, a_2) = 1$$

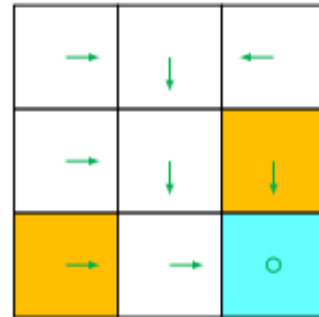
$$p(s_3|s_1, a_2) = 0$$

$$p(s_4|s_1, a_2) = 0$$

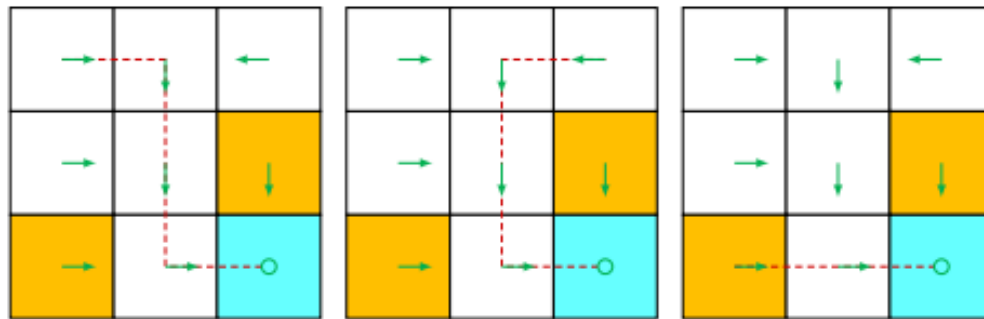
$$p(s_5|s_1, a_2) = 0$$

Działanie i Strategia

Deterministyczna strategia



(a) A deterministic policy



(b) Trajectories obtained from the policy

$$\pi(a_1|s_1) = 0$$

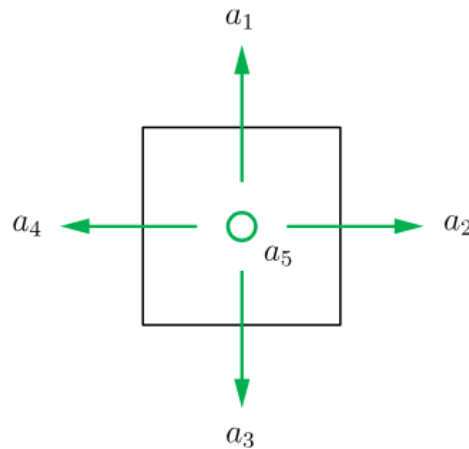
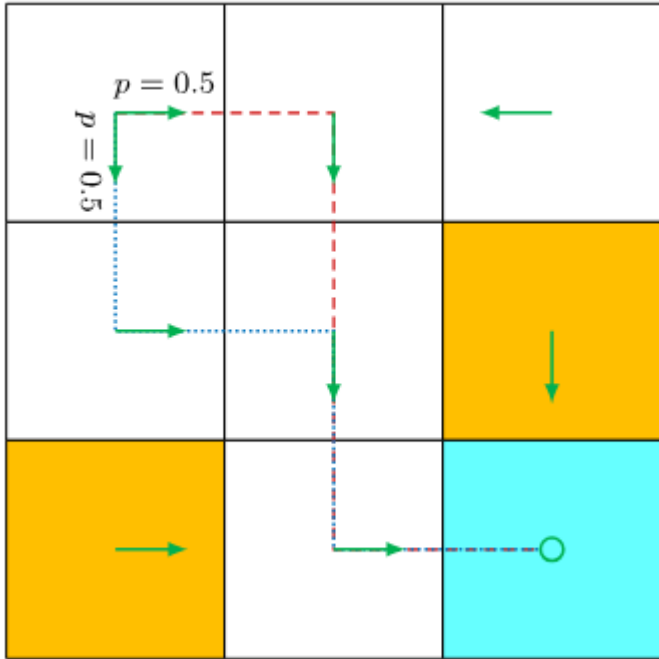
$$\pi(a_2|s_1) = 1$$

$$\pi(a_3|s_1) = 0$$

$$\pi(a_4|s_1) = 0$$

Strategia/polityka mówi agentowi jaką akcję ma wybrać w każdym stanie.

Przykład strategii stochastycznej



$$\pi(a_1|s_1) = 0$$

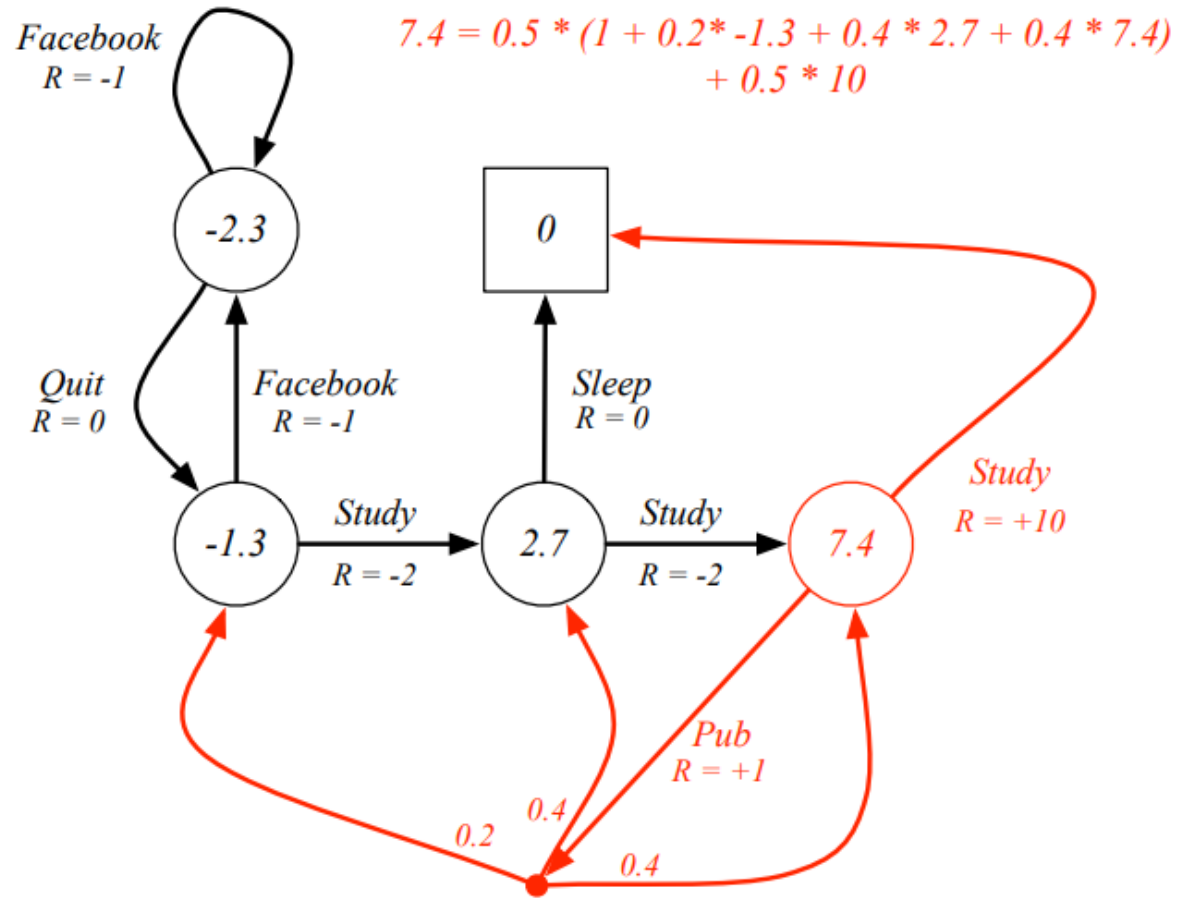
$$\pi(a_2|s_1) = 0.5$$

$$\pi(a_3|s_1) = 0.5$$

$$\pi(a_4|s_1) = 0$$

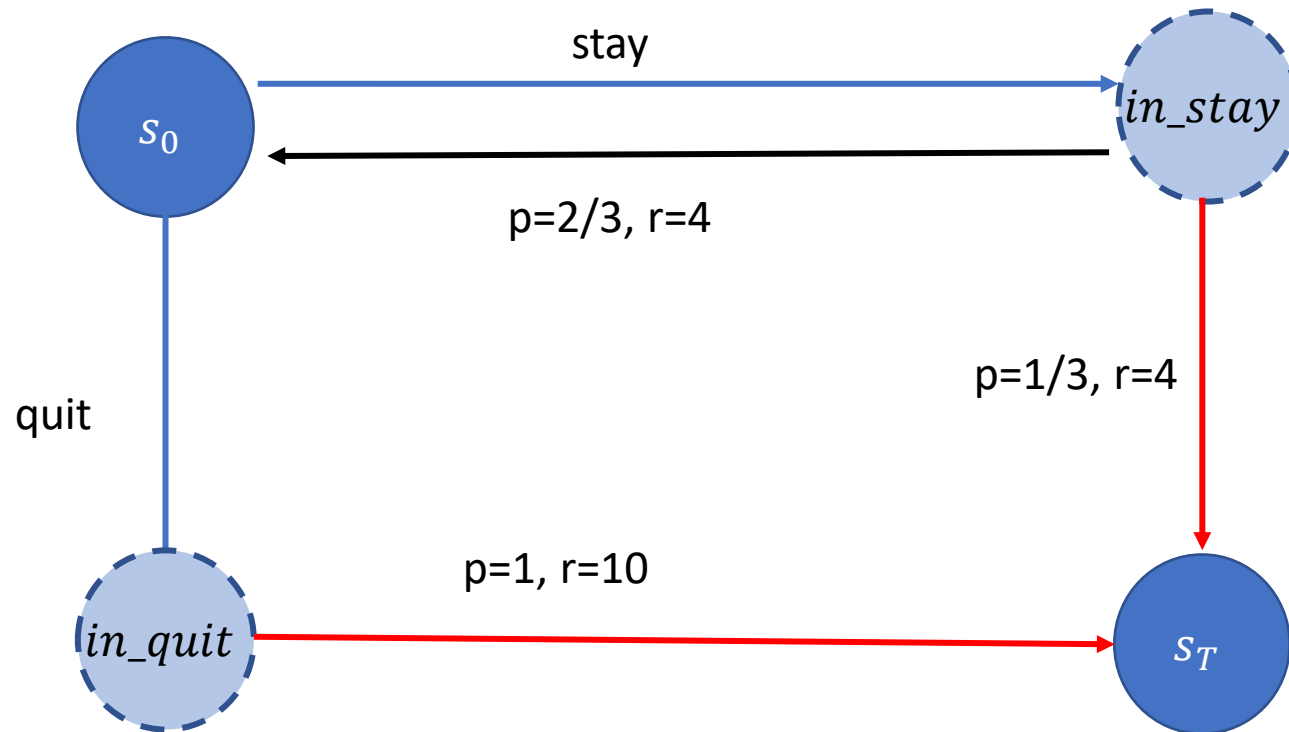
$$\pi(a_5|s_1) = 0$$

Przykład



$A = \{\text{Study, Pub}\}$

Przykład-rzut kostką



$A = \{\text{stay, quit}\}$

Po decyzji „stay” rzucamy kostką.

Jeśli wypadnie 1 lub 2 oczka przechodzimy do s_T .

W przeciwnym razie kontynuujemy-wracamy do s_0 .

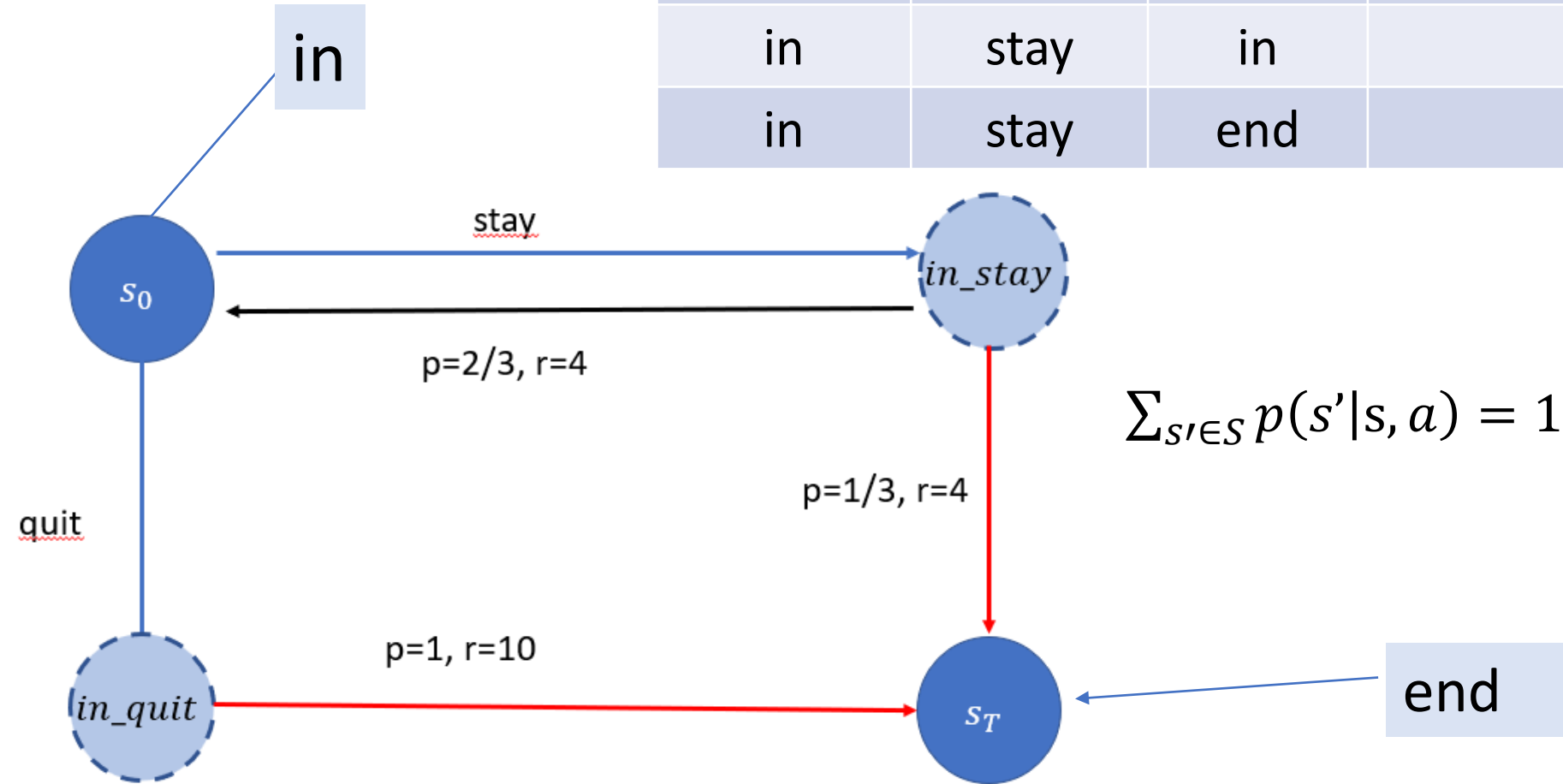
$$\gamma = 1$$

s_T -stan terminalny, koniec gry

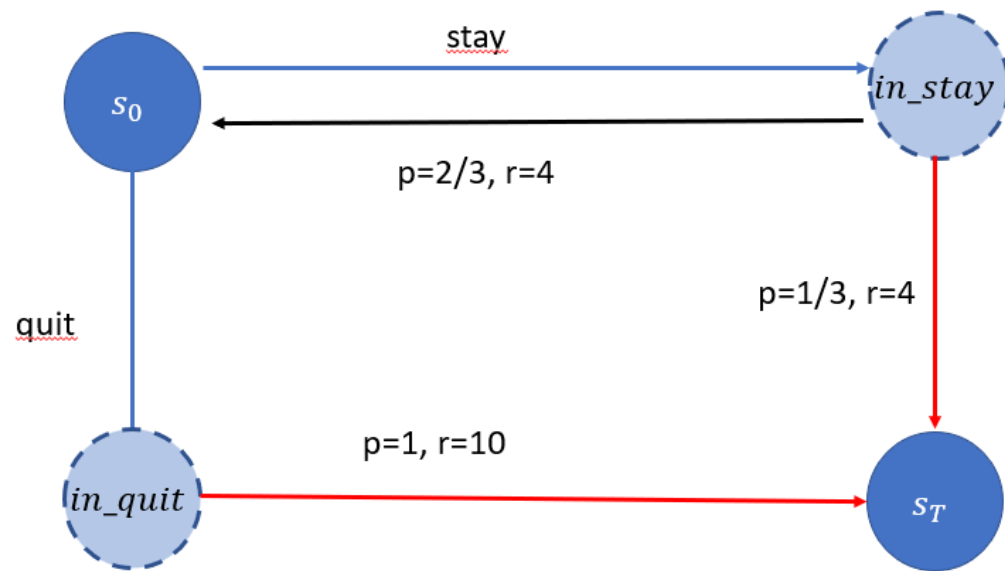
Jeśli rezygnujemy z gry, otrzymujemy $r=10$.

Jeśli zostajemy, otrzymujemy $r=4$ i rzucamy kostką.

s	a	s'	$P(s, a, s') = p(s' s, a)$
in	quit	end	1
in	stay	in	2/3
in	stay	end	1/3



$$\sum_{s' \in S} p(s'|s, a) = 1 \text{ dla ka\k{z}dej pary } (s, a)$$



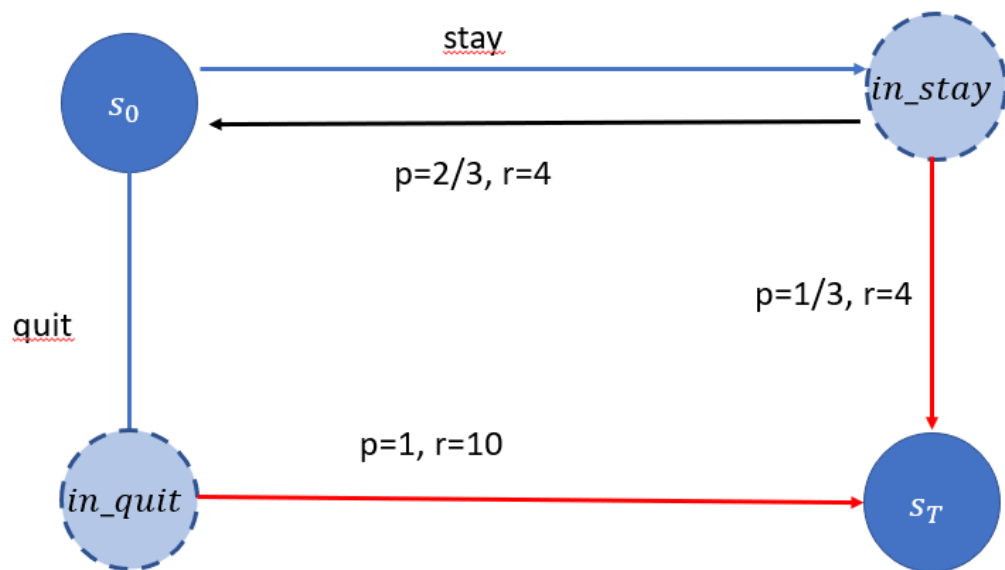
Trajektoria

Wartość

- $[s_0, \textit{stay}, 4, \textit{end}]$ 4
- $[s_0, \textit{stay}, 4, s_0, \textit{stay}, 4, s_0, \textit{stay}, 4, \textit{end}]$ 12
- $[s_0, \textit{stay}, 4, s_0, \textit{stay}, 4, \textit{end}]$ 8
- $[s_0, \textit{stay}, 4, s_0, \textit{stay}, 4, s_0, \textit{stay}, 4, s_0, \textit{stay}, 4, \textit{end}]$ 16

Związek między $V_\pi(s)$ i $Q_\pi(s, a)$

$$\bullet V_\pi(s) = \begin{cases} 0 & \text{dla } s_T \\ Q_\pi(s, \pi(s)) & \text{dla pozostałych stanów} \end{cases}$$



$V_\pi(s)$ dla $\pi(s_0) = stay$

$$V_\pi(s_0) = Q_\pi(s_0, stay)$$

$$V_\pi(s_T) = Q_\pi(s_0, stay)$$

$$V_\pi(s_T) = 0$$

$$V_\pi(s_0) = 1/3(4 + V_\pi(s_T)) + 2/3(4 + V_\pi(s_0))$$

$$V_\pi(s_0) = 12$$

Optymalna wartość funkcji stanu i stanu-akcji

- Strategia π_1 jest lepsza niż strategia π_2 jeżeli:

$$\bigwedge_{s \in S} V^{\pi_1}(s) \geq V^{\pi_2}(s)$$

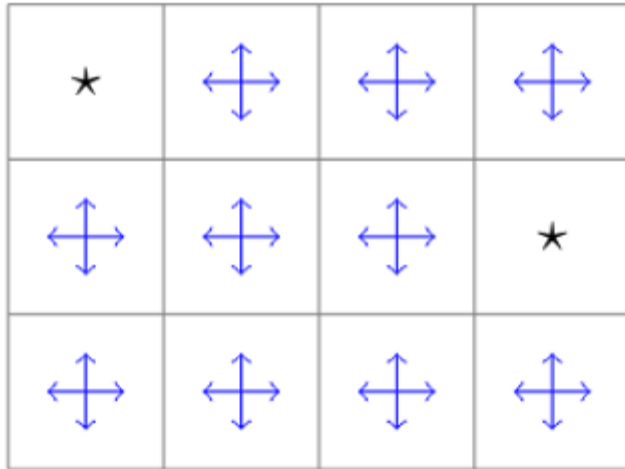
$$\bigvee_{s \in S} V^{\pi_1}(s) > V^{\pi_2}(s)$$

Strategia jest optymalna, jeżeli nie istnieje strategia od niej lepsza.

Istnieje co najmniej jedna strategia optymalna (może ich być więcej)

Wszystkie strategie optymalne mają tę samą optymalną funkcję wartości V^*

Przykład



Akcje:

1. Iść w prawo →
2. Iść w lewo ←
3. Iść w dół ↓
4. Iść w górę ↑

Nagroda: -1 za każde przejście

Cel: dotarcie do stanu terminalnego (*) w najmniejszej liczbie kroków

Strategia optymalna

★	←	↔ ↓	↓
↑	↕	→	★
↑	↕	↙	↑

Strategia optymalna

Pytania

Istnienie: Czy istnieje optymalna strategia?

Jednoznaczność: Czy optymalna strategia jest jedyna?

Stochastyczność: Czy optymalna strategia jest stochastyczna czy deterministyczna?

Algorytm: Jak uzyskać optymalną strategię i optymalne wartości stanu?

Uwaga

- Istnieje co najmniej jedna strategia optymalna.
- Wszystkie strategie optymalne mają tę samą optymalną funkcję wartości stanu V^* i optymalną funkcję wartości akcji Q^* .

Optymalna wartość funkcji stanu i funkcji Q

- $V^*(s) = \max_{\pi} V^{\pi}(s)$ dla każdego stanu $s \in S$
- $Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$ dla każdego stanu $s \in S$

Równanie optymalności Bellmana (BOE)

Równanie optymalne Bellmana opisuje wartość funkcji stanu przy optymalnej strategii, tzn. polityce, która zapewnia najwyższą możliwą wartość.

$$\begin{aligned} v^*(s) &= \max_{\pi(s) \in \Pi(s)} \sum_{a \in \mathcal{A}} \pi(a|s) \left(\sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v(s') \right) \\ &= \max_{\pi(s) \in \Pi(s)} \sum_{a \in \mathcal{A}} \pi(a|s)q(s, a), \end{aligned}$$
$$q(s, a) \doteq \sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v(s').$$

$\pi(s)$ jest strategią dla s , a $\Pi(s)$ jest zbiorem wszystkich możliwych strategii dla s .

Szukamy $v(s)$, $v'(s)$, ale nieznanymi są też $\pi(a|s)$.

Pytania

- Istnienie: Czy to równanie ma rozwiązanie?
- Jednoznaczność: Czy rozwiązanie jest jedyne?
- Algorytm: Jak rozwiązać to równanie?
- Optymalność: W jaki sposób rozwiązanie jest powiązane z optymalną strategią?

Równanie optymalności Bellmana (BOE)

Szukamy $v(s)$, $v'(s)$

BOE jest układem równań rozważanych dla każdego stanu.

Postać macierzowa jest następująca:

$$v = \max_{\pi} (r_{\pi} + \gamma P_{\pi} v),$$

gdzie $v \in R^{|S|}$, struktura r_{π} i P_{π} jest taka jak dla zwykłych równań Bellmana.

Postać macierzowa BOE

$$v = \max_{\pi} (r_{\pi} + \gamma P_{\pi} v)$$

Mamy

$v = f(v)$, gdzie f oznacza prawą stronę równania BOE

Można pokazać, że f jest odwzorowaniem zawężającym a v jest punktem stałym tego odwzorowania.

Kluczowe: $\gamma < 1$.

Z tego faktu wynika algorytm wyznaczania optymalnej wartości stanu.

Twierdzenie o istnieniu

Dla BOE postaci $v = f(v) = \max_{\pi}(r_{\pi} + \gamma P_{\pi} v)$

istnieje zawsze jednoznaczne rozwiązanie v^* , które można otrzymać iteracyjnie następująco:

$$v_{k+1} = f(v_k) = \max_{\pi}(r_{\pi} + \gamma P_{\pi} v_k) \quad k = 0, 1, 2, \dots$$

Ciąg v_k jest zbieżny do v^* z szybkością wykładniczą dla $k \rightarrow \infty$ dla dowolnej wartości początkowej v_0 .

Uwagi

Po uzyskaniu wartości v^* możemy łatwo uzyskać π^* , rozwiązując następujące zadanie

$$\pi^* = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v^*).$$

$$v^* = r_{\pi^*} + \gamma P_{\pi^*} v^*.$$

zatem $v^* = v_{\pi^*}$ jest wartością stanu π^* , a BOE jest specjalnym równaniem Bellmana, którego odpowiadającą strategią jest π^* . Rozwiązanie v^* jest optymalną wartością stanu, a π^* jest optymalną strategią.

Twierdzenie (zachłanna strategia optymalna)

Dla dowolnego $s \in S$, deterministyczna strategia zachłanna

$$\pi^*(a|s) = \begin{cases} 1, & a = a^*(s), \\ 0, & a \neq a^*(s), \end{cases}$$

jest optymalną strategią dla rozwiązywania BOE.

$$a^*(s) = \arg \max_a q^*(a, s),$$

$$q^*(s, a) \doteq \sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v^*(s').$$

Strategia zachłanna

$$a^*(s) = \arg \max_a q^*(a, s),$$

$$\pi^*(a|s) = \begin{cases} 1, & a = a^*(s), \\ 0, & a \neq a^*(s), \end{cases}$$

$$q^*(s, a) \doteq \sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v^*(s').$$

$$\pi^*(s) = \arg \max_{\pi \in \Pi} \sum_{a \in \mathcal{A}} \pi(a|s) \underbrace{\left(\sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v^*(s') \right)}_{q^*(s, a)}, \quad s \in \mathcal{S}.$$

$\sum_{a \in \mathcal{A}} \pi(a|s)q^*(s, a)$ jest maksymalizowane, jeżeli $\pi(s)$ wybiera akcję z największą wartością $q^*(s, a)$

Chociaż wartość v^* jest jedyna, optymalna strategia odpowiadająca v^* może nie być jedyna.

1. Algorytm iteracji wartości (postać macierzowa)

$$v_{k+1} = f(v_k) = \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k), \quad k = 1, 2, 3 \dots$$

Krok 1. Aktualizacja polityki/strategii, Policy Update (PU)

$$\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k), \text{ gdzie } v_k \text{ jest dane}$$

Krok 2. Aktualizacja wartości stanu

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

Krok 1. Aktualizacja polityki/strategii, Policy Update (PU)

$$\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k)$$

Można zapisać w postaci:

$$\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_k(s') \right)}_{q_k(s, a)}, s \in S$$

Po rozwiązaniu otrzymujemy optymalną strategię:

$$\pi_{k+1}(a|s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases}$$

$$a_k^*(s) = \arg \max_a q_k(a, s)$$

π_{k+1} -polityka zachłanna, bo wybiera największą wartość q .

Krok 2 Aktualizacja wartości

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

można przedstawić jako

$$v_{k+1}(s) = \sum_a \pi_{k+1}(a|s) \left(\underbrace{\sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_k(s')}_{q_k(s, a)} \right), s \in S$$

Ponieważ π_{k+1} jest zachłanna, więc powyższe można zapisać jako:

$$v_{k+1}(s) = \max_a q_k(a, s)$$

Mamy:

$$v_k(s) \rightarrow q_k(s, a) \rightarrow \text{zachłanna } \pi_{k+1}(a|s) \rightarrow \text{nowa } v_{k+1}(s) = \max_a q_k(a, s)$$

Iteracja wartości

Initialization: The probability models $p(r|s, a)$ and $p(s'|s, a)$ for all (s, a) are known.
Initial guess v_0 .

Goal: Search for the optimal state value and an optimal policy for solving the Bellman optimality equation.

While v_k has not converged in the sense that $\|v_k - v_{k-1}\|$ is greater than a predefined small threshold, for the k th iteration, do

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

q-value: $q_k(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_k(s')$

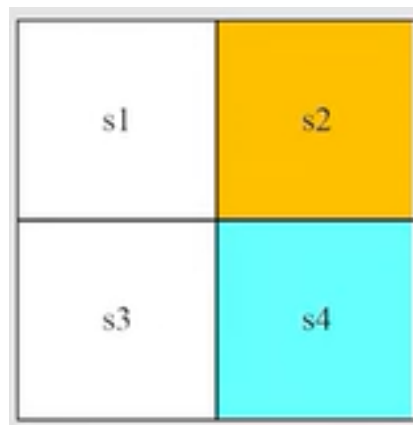
Maximum action value: $a_k^*(s) = \arg \max_a q_k(s, a)$

Policy update: $\pi_{k+1}(a|s) = 1$ if $a = a_k^*$, and $\pi_{k+1}(a|s) = 0$ otherwise

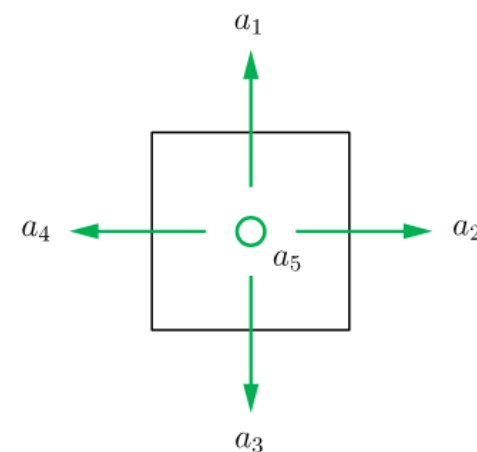
Value update: $v_{k+1}(s) = \max_a q_k(s, a)$

Przykład

Siatka z 4 stanami



$$r_{\text{boundary}} = r_{\text{forbidden}} = -1, r_{\text{target}} = 1$$



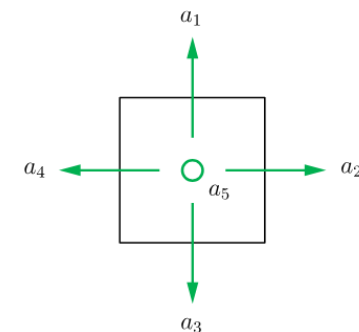
q	a1	a2	a3	a4	a5
s1	$-1 + \gamma v(s_1)$	$-1 + \gamma v(s_2)$	$0 + \gamma v(s_3)$	$-1 + \gamma v(s_1)$	$0 + \gamma v(s_1)$
s2	$-1 + \gamma v(s_2)$	$-1 + \gamma v(s_2)$	$1 + \gamma v(s_4)$	$0 + \gamma v(s_1)$	$-1 + \gamma v(s_2)$
s3	$0 + \gamma v(s_1)$	$1 + \gamma v(s_4)$	$-1 + \gamma v(s_3)$	$-1 + \gamma v(s_3)$	$0 + \gamma v(s_3)$
s4	$-1 + \gamma v(s_2)$	$-1 + \gamma v(s_4)$	$-1 + \gamma v(s_4)$	$0 + \gamma v(s_3)$	$1 + \gamma v(s_4)$

$k = 0, v_0(s_i) = 0$ dla $i = 1, 2, 3, 4$

q	a1	a2	a3	a4	a5
s1	-1	-1	0	-1	0
s2	-1	-1	1	0	-1
s3	0	1	-1	-1	0
s4	-1	-1	-1	0	1

Aktualizacja polityki

q	a1	a2	a3	a4	a5
s1	$-1 + \gamma v(s_1)$	$-1 + \gamma v(s_2)$	$0 + \gamma v(s_3)$	$-1 + \gamma v(s_1)$	$0 + \gamma v(s_1)$
s2	$-1 + \gamma v(s_2)$	$-1 + \gamma v(s_2)$	$1 + \gamma v(s_4)$	$0 + \gamma v(s_1)$	$-1 + \gamma v(s_2)$
s3	$0 + \gamma v(s_1)$	$1 + \gamma v(s_4)$	$-1 + \gamma v(s_3)$	$-1 + \gamma v(s_3)$	$0 + \gamma v(s_3)$
s4	$-1 + \gamma v(s_2)$	$-1 + \gamma v(s_4)$	$-1 + \gamma v(s_4)$	$0 + \gamma v(s_3)$	$1 + \gamma v(s_4)$



$k = 0, v_0(s_i) = 0$ dla $i = 1, 2, 3, 4$

0	0
0	0

q	a1	a2	a3	a4	a5
s1	-1	-1	0	-1	0
s2	-1	-1	1	0	-1
s3	0	1	-1	-1	0
s4	-1	-1	-1	0	1

$$\pi_1(a_5|s_1) = 1, \pi_1(a_3|s_2) = 1, \pi_1(a_2|s_3) = 1, \pi_1(a_5|s_4) = 1$$

Aktualizacja wartości

0	1
1	1

K=1

q	a1	a2	a3	a4	a5
s1	$-1 + \gamma v(s_1)$	$-1 + \gamma v(s_2)$	$0 + \gamma v(s_3)$	$-1 + \gamma v(s_1)$	$0 + \gamma v(s_1)$
s2	$-1 + \gamma v(s_2)$	$-1 + \gamma v(s_2)$	$1 + \gamma v(s_4)$	$0 + \gamma v(s_1)$	$-1 + \gamma v(s_2)$
s3	$0 + \gamma v(s_1)$	$1 + \gamma v(s_4)$	$-1 + \gamma v(s_3)$	$-1 + \gamma v(s_3)$	$0 + \gamma v(s_3)$
s4	$-1 + \gamma v(s_2)$	$-1 + \gamma v(s_4)$	$-1 + \gamma v(s_4)$	$0 + \gamma v(s_3)$	$1 + \gamma v(s_4)$

q	a1	a2	a3	a4	a5
s1	$-1 + \gamma 0$	$-1 + \gamma 1$	$0 + \gamma \cdot 1$	$-1 + \gamma \cdot 0$	$0 + \gamma \cdot 0$
s2	$-1 + \gamma \cdot 1$	$-1 + \gamma \cdot 1$	$1 + \gamma \cdot 1$	$0 + \gamma \cdot 0$	$-1 + \gamma \cdot 1$
s3	$0 + \gamma 0$	$1 + \gamma \cdot 1$	$-1 + \gamma \cdot 1$	$-1 + \gamma \cdot 1$	$0 + \gamma \cdot 1$
s4	$-1 + \gamma \cdot 1$	$-1 + \gamma$	$-1 + \gamma \cdot 1$	$0 + \gamma \cdot 1$	$1 + \gamma \cdot 1$

s1	s2
s3	s4

0	1
1	1

Krok 1 PU (aktualizacja polityki)

$$\pi_2(a_3|s_1) = 1, \pi_2(a_3|s_2) = 1, \pi_2(a_2|s_3) = 1, \pi_2(a_5|s_4) = 1$$

Krok 2 VU (aktualizacja wartości)

$\gamma \cdot 1$	$1 + \gamma \cdot 1$
$1 + \gamma \cdot 1$	$1 + \gamma \cdot 1$

2. Algorytm iteracji strategii

Inicjujemy π w sposób losowy.

Krok 1. Ewaluacja polityki/strategii, Policy Evaluation (PE)

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

v_{π_k} jest funkcją wartości stanu

Krok 2. Wyznaczenie lepszej polityki/strategii, Policy Improvement (PI)

$$\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k}), \text{ gdzie } v_{\pi_k} \text{ jest dane}$$

Algorytm prowadzi do ciągu

$$\pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI}$$

Pytania

Pytanie 1: W kroku oceny polityki, w jaki sposób uzyskać wartość stanu v_k poprzez rozwiązanie równania Bellmana?

Pytanie 2: W kroku ulepszania polityki, dlaczego nowa polityka π_{k+1} jest lepsza od π_k ?

Pytanie 3: Dlaczego taki algorytm iteracyjny może ostatecznie doprowadzić do uzyskania optymalnej polityki?

Pytanie 4: Jaki jest związek między tym algorytmem iteracji polityki a poprzednim algorytmem iteracji wartości?

Pytanie 1

W ocenie polityki, w jaki sposób uzyskać wartość stanu v_k poprzez rozwiązanie równania Bellmana

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

Przy małej liczbie stanów mamy rozwiązanie $v_{\pi_k} = (I - \gamma P_{\pi_k})^{-1} r_{\pi_k}$

Przy większej liczbie stanów rozwiązanie iteracyjne:

$$v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)} \quad j = 0, 1, 2, \dots$$

Pytanie 2

Pytanie 2: W kroku ulepszania strategii, dlaczego nowa strategia π_{k+1} jest lepsza od π_k .

Lemat (PI)

Jeżeli $\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k})$, to $v_{\pi_{k+1}} \geq v_{\pi_k}$ dla dowolnego k .

Pytanie 3

Dlaczego algorytm iteracyjny może ostatecznie doprowadzić do uzyskania optymalnej polityki?

Każda iteracja poprawia strategię:

$$v_{\pi_0} \leq v_{\pi_1} \leq v_{\pi_2} \leq \dots \leq v_{\pi_k} \leq \dots \leq v^*$$

Ciąg v_{π_k} rośnie (jest niemalejący) i jest zbieżny do v^*

Twierdzenie

Ciąg wartości stanu $(v_{\pi_k})_{k=0}^{\infty}$ generowany przez algorytm iteracji strategii jest zbieżny do optymalnej wartości stanu v^* . W rezultacie ciąg strategii π_k jest zbieżny do optymalnej polityki.

Algorytm iteracji strategii

<https://github.com/ziwenhahaha/Code-of-RL-Beginning>

Krok 1 Ewaluacja (oszacowanie) polityki/strategii

Postać macierzowa: $v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, j = 0, 1, 2, \dots$

Dla wyrazów

$$v_{\pi_k}^{(j+1)} = \sum_a \pi_k(a|s) \left(\underbrace{\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}^{(j)}(s')}_{q_{\pi_k}(s, a)} \right), s \in S$$

Kryterium stopu: $\left\| v_{\pi_k}^{(j+1)} - v_{\pi_k}^{(j)} \right\|$ $q_{\pi_k}(s, a)$ dostatecznie małe.

Krok 2. Wyznaczenie lepszej polityki/strategii, Policy Improvement (PI)

Postać macierzowa $\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k})$

Dla wyrazów:

$$\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi_k(a|s) \underbrace{\left(\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}^{(j)}(s') \right)}_{q_{\pi_k}(s, a)}$$

$q_{\pi_k}(s, a)$ jest wartością akcji dla strategii π_k .

Niech

$$a_k^*(s) = \arg \max_a q_{\pi_k}(a, s)$$

$$\pi_{k+1}(a|s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases} \quad \text{jest strategią zachłanną.}$$

Przykład

- $r_{boundary} = -1$
- $r_{cel} = 1$
- $\gamma = 0.9$
- Akcje:

a_p – w prawo,

a_l – w lewo

a_0 – bez zmian

Cel: znalezienie optymalnej strategii przez iterację strategii.



Iteracja strategii

1. Iteracja $k=0$: krok 1 Ewaluacja strategii

π_0 wybrane jak na rysunku po prawej.

Możemy zapisać równania Bellmana

$$v_{\pi_0}(s_1) = -1 + \gamma v_{\pi_0}(s_1)$$

$$v_{\pi_0}(s_2) = 0 + \gamma v_{\pi_0}(s_1)$$

I rozwiązać układ bezpośrednio:

$$\begin{pmatrix} 1 - \gamma & 0 \\ 1 & -\gamma \end{pmatrix} \begin{pmatrix} v_{\pi_0}(s_1) \\ v_{\pi_0}(s_2) \end{pmatrix} = \begin{pmatrix} -1 \\ 0 \end{pmatrix} \text{ dla } \gamma = 0.9$$

$$v_{\pi_0}(s_1) = -10, v_{\pi_0}(s_2) = -9$$



Algorytm iteracji strategii

Równanie można rozwiązać też iteracyjnie

$$\begin{aligned}v_{\pi_0}(s_1) &= -1 + \gamma v_{\pi_0}(s_1) \\ v_{\pi_0}(s_2) &= 0 + \gamma v_{\pi_0}(s_1)\end{aligned}$$

Wtedy początkowe wartości stanów ustawiamy np. na 0 $v_{\pi_0}^{(0)}(s_1) = 0$

$v_{\pi_0}^{(0)}(s_2) = 0$ i mamy:

$$\begin{aligned}v_{\pi_0}^{(1)}(s_1) &= -1 + \gamma v_{\pi_0}^{(0)}(s_1) = -1 \\ v_{\pi_0}^{(1)}(s_2) &= 0 + \gamma v_{\pi_0}^{(0)}(s_1) = 0\end{aligned}$$

Algorytm iteracji strategii

$$\begin{aligned}v_{\pi_0}^{(1)}(s_1) &= -1 + \gamma v_{\pi_0}^{(0)}(s_1) = -1 \\v_{\pi_0}^{(1)}(s_2) &= 0 + \gamma v_{\pi_0}^{(0)}(s_1) = 0\end{aligned}$$

$$\begin{aligned}v_{\pi_0}^{(2)}(s_1) &= -1 + \gamma v_{\pi_0}^{(1)}(s_1) = -1.9 \\v_{\pi_0}^{(2)}(s_2) &= 0 + \gamma v_{\pi_0}^{(1)}(s_1) = -0.9\end{aligned}$$

Algorytm iteracji strategii

- Iteracja $k=0$, krok 2. PI (poprawa strategii)

Budujemy q-tablicę: $q_{\pi_k}(s, a)$



$q(s,a)$	a_l	a_0	a_p
s_1	$-1 + \gamma v_{\pi_k}(s_1)$	$0 + \gamma v_{\pi_k}(s_1)$	$1 + \gamma v(s_2)$
s_2	$0 + \gamma v_{\pi_k}(s_1)$	$1 + \gamma v_{\pi_k}(s_2)$	$-1 + \gamma v_{\pi_k}(s_4)$

a_p – w prawo,
 a_l – w lewo
 a_0 – bez zmian

Podstawiamy: $v_{\pi_0}(s_1) = -10$, $v_{\pi_0}(s_2) = -9$, $\gamma = 0.9$

$q(s,a)$	a_l	a_0	a_p
s_1	-10	-9	-7.1
s_2	-9	-7.1	-9.1

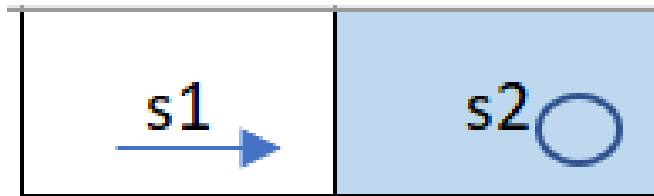
Algorytm iteracji strategii

q(s,a)	al.	a0	ap
s1	-10	-9	-7.1
s2	-9	-7.1	-9.1

Szukamy największych wartości w q-tabeli, aby poprawić strategię.

Poprawiona strategia:

$\pi_1(a_p|s_1) = 1, \pi_1(a_0|s_2) = 1$
jest już optymalna.



Algorytm iteracji strategii

Initialization: The system model, $p(r|s, a)$ and $p(s'|s, a)$ for all (s, a) , is known. Initial guess π_0 .

Goal: Search for the optimal state value and an optimal policy.

While v_{π_k} has not converged, for the k th iteration, do

Policy evaluation:

Initialization: an arbitrary initial guess $v_{\pi_k}^{(0)}$

While $v_{\pi_k}^{(j)}$ has not converged, for the j th iteration, do

For every state $s \in \mathcal{S}$, do

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}^{(j)}(s') \right]$$

Policy improvement:

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}$, do

$$q_{\pi_k}(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}(s')$$

$$a_k^*(s) = \arg \max_a q_{\pi_k}(s, a)$$

$$\pi_{k+1}(a|s) = 1 \text{ if } a = a_k^*, \text{ and } \pi_{k+1}(a|s) = 0 \text{ otherwise}$$

Podsumowanie

Algorytm iteracji wartości jest algorytmem iteracyjnym do rozwiązywania BOE (równań optymalnych Bellmana).

$$v_{k+1} = f(v_k) = \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k), \quad k = 1, 2, 3 \dots$$

$$\begin{cases} PU: \pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k) \\ VU: v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k \end{cases}$$

Podsumowanie

Algorytm iteracji strategii, dla danej strategii π_0 .

$$\begin{cases} PE: v_k = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k} \\ VU: \pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k}) \end{cases}$$

Policy iteration: $\pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \dots$

Value iteration: $u_0 \xrightarrow{PU} \pi'_1 \xrightarrow{VU} u_1 \xrightarrow{PU} \pi'_2 \xrightarrow{VU} u_2 \xrightarrow{PU} \dots$

Porównanie

Przyjrzyjmy się bliżej ich krokom dla wartości, aby zobaczyć różnicę między tymi dwoma algorytmami.

W szczególności, niech oba algorytmy rozpoczną się od tego samego warunku początkowego:

$$v_0 = v_{\pi_0}$$

	Policy iteration algorithm	Value iteration algorithm	Comments
1) Policy:	π_0	N/A	
2) Value:	$v_{\pi_0} = r_{\pi_0} + \gamma P_{\pi_0} v_{\pi_0}$	$v_0 := v_{\pi_0}$	
3) Policy:	$\pi_1 = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_0})$	$\pi_1 = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_0)$	The two policies are the same
4) Value:	$v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$	$v_1 = r_{\pi_1} + \gamma P_{\pi_1} v_0$	$v_{\pi_1} \geq v_1$ since $v_{\pi_1} \geq v_{\pi_0}$
5) Policy:	$\pi_2 = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_1})$	$\pi'_2 = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_1)$	
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Rozwiązanie równania dla $k=1$

$$v_k = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

		$v_{\pi_1}^{(0)} = v_0$
Iteracja wartości	$\leftarrow v_1 \leftarrow$	$v_{\pi_1}^{(1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)}$
		$v_{\pi_1}^{(2)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)}$
		$\vdots$
Obcięta iteracja strategii	$\leftarrow \bar{v}_1 \leftarrow$	$v_{\pi_1}^{(j)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)}$
		$\vdots$
Iteracja strategii	$\leftarrow v_{\pi_1} \leftarrow$	$v_{\pi_1}^{(\infty)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)}$

	$v_{\pi_1}^{(0)} = v_0$	Obcięta iteracja strategii	$\leftarrow \bar{v}_1 \leftarrow v_{\pi_1}^{(j)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)}$
Iteracja wartości	$\leftarrow v_1 \leftarrow v_{\pi_1}^{(1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)}$		$\vdots$
	$v_{\pi_1}^{(2)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)}$	Iteracja strategii	$\leftarrow v_{\pi_1} \leftarrow v_{\pi_1}^{(\infty)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)}$
	$\vdots$		

Z powyższego procesu można wyciągnąć następujące wnioski.

- Jeśli iteracja jest wykonywana tylko raz, to $v_{\pi_1}^{(1)}$ jest równe v_1 (mamy algorytm VI).
- Jeśli iteracja jest wykonywana ∞ wiele razy, to mamy algorytm iteracji strategii $v_{\pi_1}^{(\infty)} = v_{\pi_1}$
- Jeśli iteracja jest wykonywana n-krotnie (oznaczane jako $j_{truncate}$), wówczas taki algorytm
- nazywany jest obciętą iteracją polityki.

Nazywa się go obciętym, ponieważ pozostałe iteracje od $j_{truncate}$ do ∞ są obcinane.

W rezultacie algorytmy iteracji wartości i iteracji polityki można postrzegać jako dwa skrajne przypadki algorytmu obciętej iteracji polityki: iteracja wartości kończy się na $j_{truncate} = 1$

Zaś algorytm iteracji strategii na $j_{truncate} = \infty$. Porównanie możliwe przy założeniu

$$v_0 = v_{\pi_0}$$

Algorytm TPI (obcięta iteracja strategii)

Initialization: The probability models $p(r|s, a)$ and $p(s'|s, a)$ for all (s, a) are known.

Initial guess π_0 .

Goal: Search for the optimal state value and an optimal policy.

While v_k has not converged, for the k th iteration, do

Policy evaluation:

Initialization: select the initial guess as $v_k^{(0)} = v_{k-1}$. The maximum number of iterations is set as j_{truncate} .

While $j < j_{\text{truncate}}$, do

For every state $s \in \mathcal{S}$, do

$$v_k^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_k^{(j)}(s') \right]$$

Set $v_k = v_k^{(j_{\text{truncate}})}$

Policy improvement:

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

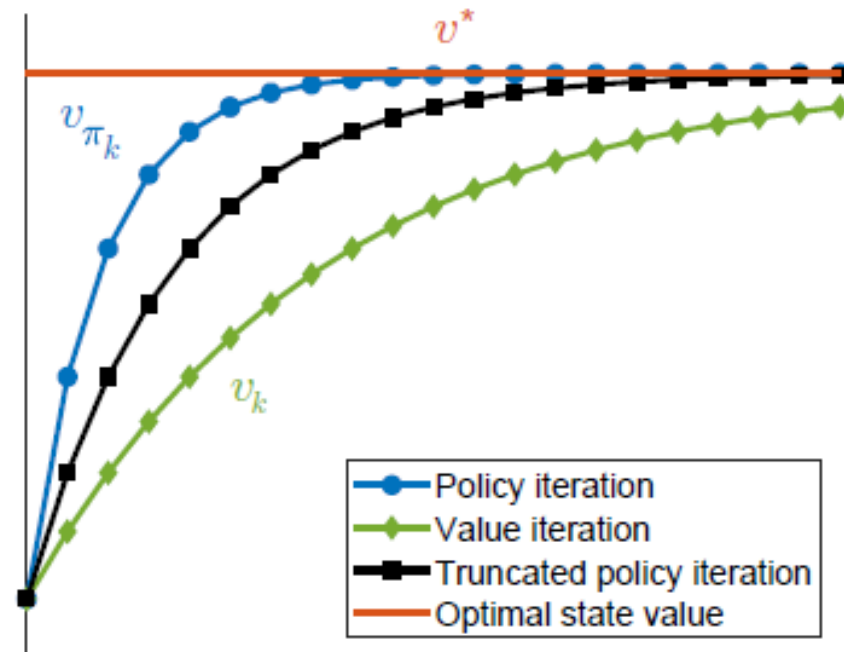
$$q_k(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_k(s')$$

$$a_k^*(s) = \arg \max_a q_k(s, a)$$

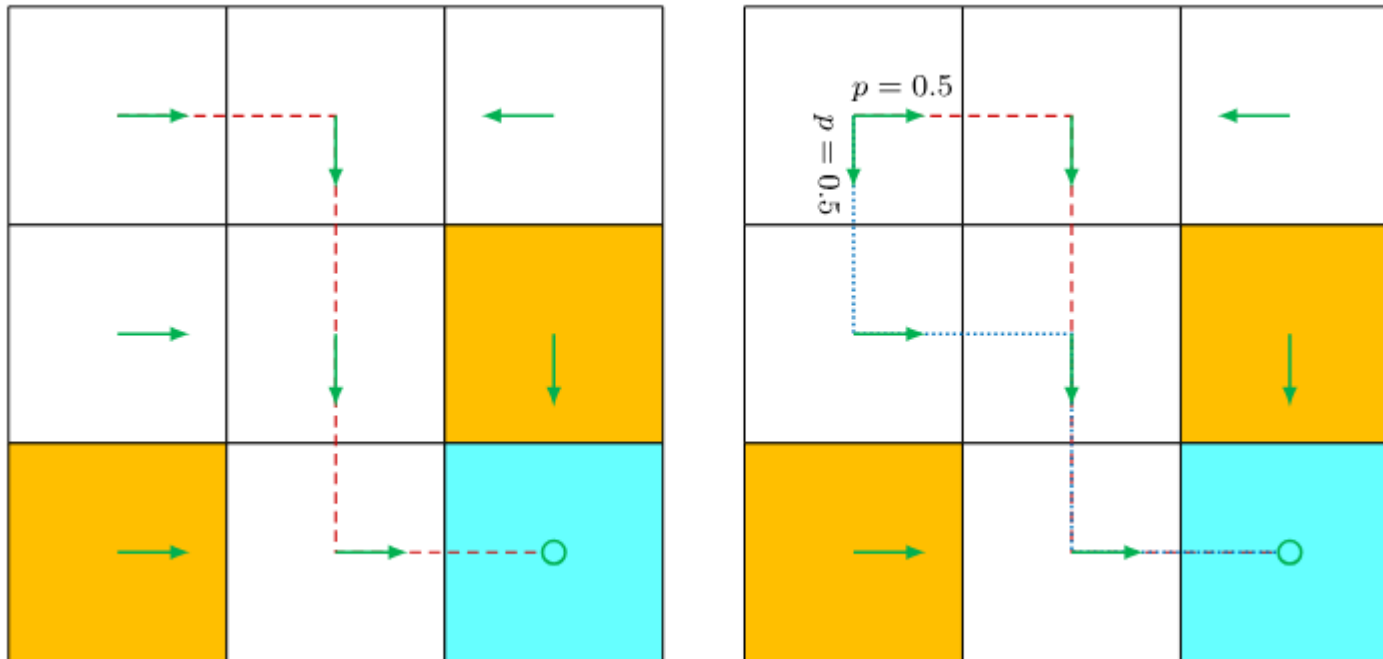
$$\pi_{k+1}(a|s) = 1 \text{ if } a = a_k^*, \text{ and } \pi_{k+1}(a|s) = 0 \text{ otherwise}$$

Algorytm TPI (obcięta iteracja strategii)

Algorytmy iteracji wartości i iteracji polityki są dwoma szczególnymi przypadkami algorytmu obciętej iteracji polityki.



Przykład



Optymalna strategia może nie być jedyna. Zawsze istnieje optymalna strategia deterministyczna.

Wyznaczenie $q(a,s)$ (wartości akcji) bez modelu

$$q_{\pi_k}(s, a) = E(G_t | S_t = s, A_t = a) \approx \frac{1}{n} \sum_{i=1}^n g_{\pi_k}^{(i)}(s, a)$$

gdzie $g_{\pi_k}^{(i)}(s, a)$ jest nagrodą otrzymaną z realizacji i -tego epizodu

Jeśli nie mamy modelu, wykorzystujemy dane - dane to **doświadczenie, experience.**

MC ε -Greedy

$$\pi(a|s) = \begin{cases} 1 - \frac{\varepsilon}{|A(s)|} (|A(s)| - 1) & \text{dla akcji zachłannej} \\ \frac{\varepsilon}{|A(s)|} & \text{dla pozostałych } |A(s)| - 1 \text{ akcji} \end{cases}$$

$|A(s)|$ oznacza liczbę akcji dla stanu s

Dla $\varepsilon = 0$, algorytm ε – greedy staje się zachłannym. Dla $\varepsilon = 1$ prawdopodobieństwo wyboru dowolnej akcji jest takie samo $\frac{1}{|A(s)|}$

Eksploracja vs. Eksploatacja

Jak odkryć dobre działania?

Odpowiedź:

Musimy zbadać przestrzeń stanów/akcji.

W ten sposób RL łączy w sobie dwa zadania:

Eksploracja:

Wypróbuj nową akcję a w stanie s , obserwuj nagrodę r_t .

Odkrywa więcej informacji o środowisku, ale poświęca całkowitą nagrodę.

Przykład gry: Rozegranie nowego eksperymentalnego ruchu.

Eksploatacja:

Wykorzystanie wcześniej odkrytej dobrej akcji

Wykorzystanie znanych informacji w celu maksymalizacji nagrody, ale poświęcenie niezbadanych obszarów

Przykład gry: Zagraj ruch, który uważasz za najlepszy

Kompromis: Ważne jest, aby jednocześnie odkrywać i wykorzystywać. Na początku eksploracja.

Eksploracja vs. Eksploatacja

Jak zrównoważyć eksplorację i eksploatację?

Zachłanny algorytm eksploracji:

- Wypróbuj wszystkie możliwe działania z niezerowym prawdopodobieństwem
- Z prawdopodobieństwem ε wybierz działanie losowo (eksploracja)
- Z prawdopodobieństwem $1 - \varepsilon$ wybierz najlepsze działanie (eksploatacja)

Działanie zachłanne jest definiowane jako najlepsze działanie, które zostało odkryte do tej pory

ε jest początkowo duże i stopniowo zmniejszane w czasie.

TD-metoda różnic czasowych

- Bez modelu;
- Zawiera elementy MC i DP (programowanie dynamiczne);
- Polega na natychmiastowej aktualizacji wartości stanu po każdej realizacji (s , a , r , s');

<https://www.geeksforgeeks.org/deep-learning/temporal-difference-td-learning/>

Dane niezbędne do obliczeń w TD

Trajektoria $(s_0, r_1, s_1, \dots, s_t, r_{t+1}, s_{t+1}, \dots)$ lub $\{s_t, r_{t+1}, s_{t+1}\}_t$
generowana przez strategię π

Cel: estymacja $v_\pi(s)$

Algorytm TD aktualizacja oszacowania wartości stanu po każdej obserwacji (s, a, r, s')

$$\begin{cases} v_{t+1}(s_t) = v_t(s_t) - \alpha_k(s_t)[v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))] \\ v_{t+1}(s) = v_t(s), & \text{dla każdego } s \neq s_t \end{cases}$$

gdzie $t = 0, 1, 2, 3, \dots$, $v_t(s_t)$ jest oszacowaną wartością stanu $v_\pi(s_t)$;
 $\alpha_k(s_t)$ jest stałą uczenia dla stanu s_t w chwili t .

W chwili t można odwiedzić tylko jeden stan.

Nawiązanie do równań Bellmana

Mamy definicję wartości stanu

$$v_{\pi}(s) = E[R + \gamma G | S = s], \quad s \in S$$

gdzie G jest zdyskontowaną nagrodą

$$E[G | S = s] = \sum_a \pi(a|s) \sum_{s'} p(s'|s, a) v_{\pi}(s') = E[v_{\pi}(s') | S = s],$$

gdzie s' jest kolejnym stanem.

Otrzymujemy wersję równania Bellmana tzw. Bellman Expectation equation:

$$v_{\pi}(s) = E[R + \gamma v_{\pi}(s') | S = s], \quad s \in S$$

Algorytm EM dla BEE (Bellman Expectation Equation)

$$v_{\pi}(s) = E[R + \gamma v_{\pi}(s') | S = s], s \in S$$

$$g(v(s)) = v(s) - E[R + \gamma v_{\pi}(s') | s], s \in S$$

$$g(v(s)) = 0$$

$$\begin{aligned} \tilde{g}(v(s)) &= v(s) - [r + \gamma v(s')] = (v(s) - E[R + \gamma v_{\pi}(s') | s]) + (E[R + \\ &\gamma v_{\pi}(s') | s] - [r + \gamma v(s')]) = g(v(s)) + \eta \end{aligned}$$

Algorytm TD (Różnic czasowych)

W algorytmie TD szacujemy wartości stanów dla danej strategii.

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t \tilde{g}(v_t(s)) = v_t(s_t) - \alpha_t (s_t) [v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))]$$

$$\boxed{v_{t+1}(s_t)} = \underbrace{v_t(s_t)}_{\text{Wartość bieżąca}} - \alpha_t (s_t) \underbrace{[v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))]}_{\text{TD error (błąd)}}$$

Nowe oszacowanie

$\bar{v}_t = r_{t+1} + \gamma v_t(s_{t+1})$ nazywa się **TD target**

$$v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1})) = v_t(s_t) - \bar{v}_t$$

Algorytm TD(0)

Szacuje wartość stanu dla danej strategii, nie szacuje wartości akcji

TD(0)-jeden krok

Zainicjuj strategię π oraz $v_\pi(s) = 0$, dla każdego $s \in S$.

Powtarzaj:

- Wylosuj (s_t, r_{t+1}, s_{t+1})

$$v_{t+1}(s_t) = v_t(s_t) + \alpha_t [(r_{t+1} + \gamma v_t(s_{t+1}) - v_t(s_t))]$$

$$v_{t+1}(s_t) = v_t(s_t) \quad \wedge_s s \neq s_t$$

Q-learning

Jeśli nie znamy modelu algorytm Q-learning działa także według zasady Bellmana:

Iteracyjne wyznaczanie Q^* : $Q^*(s_t, a_t) = \mathbb{E} \left[r_t + \gamma \max_{a' \in \mathcal{A}} Q^*(s_{t+1}, a') \middle| s_t, a_t \right]$
 (s_t, a_t) – stan i akcja w momencie t

wyznaczamy aktualizowany ciąg $Q_1, Q_2, \dots$ ze stałą uczenia α :

$$\begin{aligned} Q_{i+1}(s_t, a_t) &\leftarrow (1 - \alpha)Q_i(s_t, a_t) + \alpha(r_t + \gamma \max_{a' \in \mathcal{A}} Q_i(s_{t+1}, a')) \\ &= Q_i(s_t, a_t) + \underbrace{\alpha(r_t + \gamma \max_{a' \in \mathcal{A}} Q_i(s_{t+1}, a') - Q_i(s_t, a_t))}_{\text{cel}} \end{aligned}$$

Q_i jest zbieżny do Q^*

TD-temporal difference error/błąd różnic czasowych

Strategia wyznaczana przy pomocy tablic Q

$$\underline{Q_{i+1}(s_t, a_t)} \leftarrow \underline{Q_i(s_t, a_t)} + \alpha(\underbrace{r_t}_{\text{Nagroda natychmiastowa}} + \underbrace{\gamma \max_{a' \in A} Q_i(s_{t+1}, a')}_{\text{Zdyskontowane oszacowanie optymalnej Q-value następnego stanu}} - \underline{Q_i(s_t, a_t)})$$

czynnik dyskontujący

Zdyskontowane oszacowanie optymalnej Q-value następnego stanu

Poprzednia wartość Q

Nagroda natychmiastowa

Poprzednia wartość Q

TD

błąd TD

Q-learning

Algorytm ten w środowisku epizodycznym implementowany jest w następujący sposób:

1. Stwórz **tablicę wartości** dla każdego stanu i powiązanych z nim możliwych akcji.
2. Ustal liczbę epizodów.
3. Dopóki nie zostanie osiągnięty **stan końcowy**:
 - **wybierz** akcję wedle obranej strategii,
 - **wykonaj** akcję a i **obserwuj** nagrodę r i kolejny stan s' ,
 - zaktualizuj **wartość oczekiwanej nagrody** według wzoru:
 $Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a) - Q(s, a))$.
4. Jeżeli został osiągnięty stan końcowy, przejdź do **kolejnego epizodu**.

Q-learning

- Initialize Q table and initial state s_0 randomly
- Repeat:
 - Observe state s_t , choose action a_t according to ϵ -greedy strategy
 - Observe reward r_t and next state s_{t+1}
 - Compute TD error: $r_t + \gamma \max_{a' \in \mathcal{A}} Q_i(s_{t+1}, a') - Q_i(s_t, a_t)$
 - Update Q-table

Uczenie się optymalnej strategii dla SARSA

Initialization: $\alpha_t(s, a) = \alpha > 0$ for all (s, a) and all t . $\epsilon \in (0, 1)$. Initial $q_0(s, a)$ for all (s, a) . Initial ϵ -greedy policy π_0 derived from q_0 .

Goal: Learn an optimal policy that can lead the agent to the target state from an initial state s_0 .

For each episode, do

Generate a_0 at s_0 following $\pi_0(s_0)$

If s_t ($t = 0, 1, 2, \dots$) is not the target state, do

Collect an experience sample $(r_{t+1}, s_{t+1}, a_{t+1})$ given (s_t, a_t) : generate r_{t+1}, s_{t+1} by interacting with the environment; generate a_{t+1} following $\pi_t(s_{t+1})$.

Update q -value for (s_t, a_t) :

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})) \right]$$

Update policy for s_t :

$$\pi_{t+1}(a|s_t) = 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|} (|\mathcal{A}(s_t)| - 1) \text{ if } a = \arg \max_a q_{t+1}(s_t, a)$$

$$\pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}(s_t)|} \text{ otherwise}$$

$$s_t \leftarrow s_{t+1}, a_t \leftarrow a_{t+1}$$

Deep Q-learning (DQN)

Wykorzystujemy spadek gradientowy do rozwiązania zadania minimalizacji funkcji straty:

$$J = \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w) \right)^2 \right],$$

$$\nabla_w J = -\mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w) \right) \nabla_w \hat{q}(S, A, w) \right],$$

$$y_T = r + \gamma \max_a \hat{q}(s', a, w_T)$$

Dwie sieci w DQN

- $\hat{q}(s, a, w)$ -sieć główna (main network)
- $\hat{q}(s, a, w_T)$ -sieć docelowa (target network)

$$J(w) = \mathbb{E} \left[(R + \gamma \max_{a \in A(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w))^2 \right]$$

w_T -parametr sieci targetowej. Dla ustalonego w_T obliczamy gradient J względem w :

$$\nabla_w J = \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w) \right) \nabla_w \hat{q}(S, A, w) \right]$$

Deep Q-learning (DQN)

Wartości wag w_T są aktualizowane poprzez zastąpienie ich wyznaczonymi wagami dla sieci głównej.

Wagi sieci głównej są optymalizowane przy stałych wagach w_T .

Z czasem oba zestawy wag w_T i w dążą do wartości optymalnych.

Wykorzystywany jest algorytm spadku gradientowego GD dla sieci głównej.

Opis algorytmu

Wstępnie w_T i w są takie same. W każdej iteracji losujemy mini-paczkę (mini-wsad) próbek danych $\{s, a, r, s'\}$ (mini-batch) z **replay buffer**.

Mini-batch to niewielka podgrupa danych treningowych, która jest wykorzystywana do obliczenia jednej aktualizacji wag modelu w procesie uczenia. Zamiast używać:

- *całego zbioru danych* (batch gradient descent),
- *lub pojedynczego przykładu* (stochastic gradient descent),

stosuje się mini-batch gradient descent, gdzie dane są dzielone na małe porcje (np. 32, 64, 128 próbek).

Epoka = wiele iteracji (tyle, ile mini-batchy w całym zbiorze danych)

Deep Q-learning (DQN)

- W każdej iteracji losujemy mini-paczkę (mini-wsad) próbek danych $\{s, a, r, s'\}$ (mini-batch) z **replay buffer**.
- Wylosowane (s, a) wykorzystujemy do obliczenia

$$y_T = r + \gamma \max_a \hat{q}(s', a, w_T)$$

- Następnie minimalizujemy TD error (funkcję straty)

$(y_T - \hat{q}(s, a, w))^2$ wykorzystując mini-batch $\{(s, a, y_T)\}$, czyli wyznaczam wagi sieci głównej.

Aktualizujemy w_T

Deep Q-learning (DQN)

Experience replay to proces przechowywania i losowania doświadczenia agenta w postaci tzw. transition tuples, czyli: (s_t, a_t, r_t, s_{t+1}) ze zbioru **Replay buffer**, gdzie:

- s_t – stan w chwili t ,
- a_t – akcja podjęta w tym stanie,
- r_t – nagroda otrzymana po wykonaniu akcji,
- s_{t+1} – nowy stan po wykonaniu akcji.

Replay buffer (zbiór próbek) pozwala losowo pobierać próbki, co zmniejsza korelację i stabilizuje uczenie.

Efektywne wykorzystanie doświadczeń: Zamiast odrzucać stare doświadczenia, można je wykorzystać wielokrotnie.

Batch learning: Umożliwia uczenie na mini-batchach, co poprawia wydajność i stabilność.

Alorytm DQN

Initialization: A main network and a target network with the same initial parameter.

Goal: Learn an optimal target network to approximate the *optimal* action values from the experience samples generated by a given behavior policy π_b .

Store the experience samples generated by π_b in a replay buffer $\mathcal{B} = \{(s, a, r, s')\}$

For each iteration, do

Uniformly draw a mini-batch of samples from $\mathcal{B}$

For each sample (s, a, r, s') , calculate the target value as $y_T = r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$, where w_T is the parameter of the target network

Update the main network to minimize $(y_T - \hat{q}(s, a, w))^2$ using the mini-batch of samples

Set $w_T = w$ every C iterations

$$y_T = r + \gamma \max_a \hat{q}(s', a, w_T)$$

Deep Q-learning (DQN)

1. Agent zbiera doświadczenia podczas interakcji ze środowiskiem.
2. Każde doświadczenie trafia do bufora (zwykle o ograniczonej pojemności).
3. Podczas aktualizacji sieci neuronowej losowo wybierane są próbki z bufora.

Strona Koła Naukowego Main

Instytut Informatyki Stosowanej
Politechnika Łódzka

<https://main.p.lodz.pl/news.php?id=124>

Q-Learning i SARSA - bez ryzyka nie ma zabawy

Q-Learning i SARSA

Table 1. The different parameters of the states and their possible values.

Parameter	Possible Values	Comment
Location	$\{port, turbine\}$	
Wave height [m]	$\{0, 0.4, 0.8, \dots, 10, 10.4\}$	
Repair time [h]	$\{0, 1, 2, \dots, max - 1, max\}$	maximum (max) depends on component
Steps waited	$\{0, 1, 2, 3\}$	depends on strategy

Table 2. The different actions and how they influence the parameters of the next state.

Action	Parameters for Next State
stay	
wait	steps waited +1
reset wait time	steps waited = 0
go out	location = turbine
repair	repair time −1
return	location = port

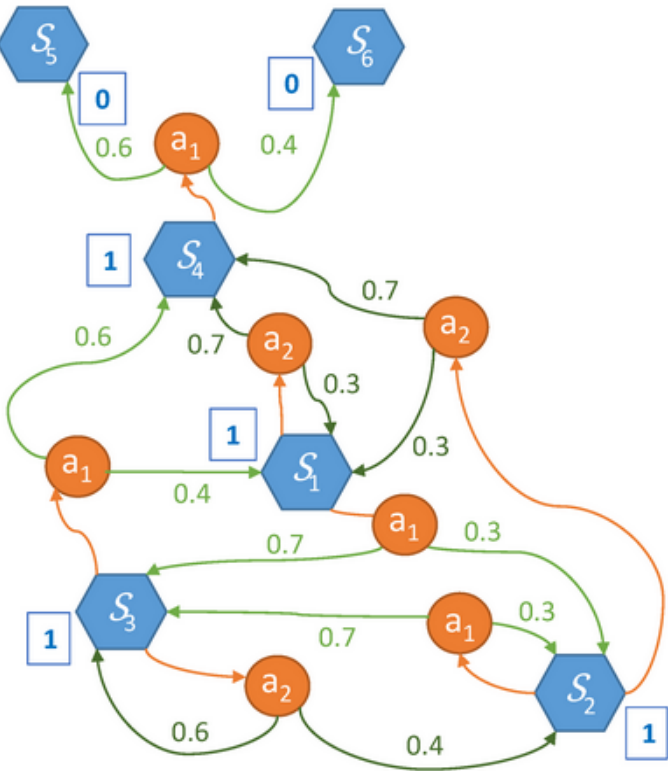


Article

Use of Markov Decision Processes in the Evaluation of Corrective Maintenance Scheduling Policies for Offshore Wind Farms

Helene Seyr  and Michael Muskulus 

Figure 1. An example of a Markov decision process (left). The blue hexagons represent the states, with their rewards indicated in blue boxes next to the state. Orange circles indicate the two actions that can be taken in each state. Subject to the action, the transition probabilities are indicated with green arrows and the value displayed next to the arrow. Transition probabilities following action a_1 are shown in light green, while transition probabilities following action a_2 are shown in dark green. The policies “choose action a_1 in all states” (upper) and “choose action a_2 in all states” (lower) are presented by the transition matrices (right).



$$\pi(S_i) = a_1$$

	s_1	s_2	s_3	s_4	s_5	s_6
s_1	0	0.3	0.7	0	0	0
s_2	0	0.3	0.7	0	0	0
s_3	0.4	0	0	0.6	0	0
s_4	0	0	0	0	0.6	0.4
s_5	0	0	0	0	0	0
s_6	0	0	0	0	0	0

$$\pi(S_i) = a_2$$

	s_1	s_2	s_3	s_4	s_5	s_6
s_1	0.3	0	0	0.7	0	0
s_2	0.3	0	0	0.7	0	0
s_3	0	0.4	0.6	0	0	0
s_4	0	0	0	0	0	0
s_5	0	0	0	0	0	0
s_6	0	0	0	0	0	0

Ćwiczenia

https://gymnasium.farama.org/environments/toy_text/frozen_lake/